

DOSSIER D'ARCHITECTURE
TECHNIQUE

InduSense 4.0

Sprint 3 CISIA

Industrialiser · exposer · orchestrer ·
superviser

API	PREFECT	CI/CD	OBSERVABILITÉ
-----	---------	-------	---------------

DOCUMENT DE RÉFÉRENCE

Architecture fonctionnelle complète · modules 23 à 34 · quatre schémas autonomes et éditables.

Version 1.0
Date 4 septembre 2026
Périmètre modules 23 à 34
Statut architecture de référence fonctionnelle

■ Sommaire

01 Gestion documentaire

10 Observabilité, drift et exploitation

02 Résumé exécutif

11 Sécurité et frontières de confiance

03 Périmètre et hypothèses

12 Déploiements et environnements

04 Vue globale du Sprint 3 complet

13 Tests et critères de recette

05 Architecture détaillée de l'API

14 Risques et mesures de maîtrise

06 Fonctionnement de Prefect

15 Décisions d'architecture

07 Données, modèle et persistance

16 Traçabilité pédagogique modules 23 à 34

08 Conteneurisation et exécution locale

17 Glossaire

09 CI/CD et stratégie de promotion

18 Sources et provenance

LÉGENDE

Teal = composant fonctionnel · bleu pâle = donnée ou artefact · gris clair = entrée ou support · rouge sombre = contrôle ou action critique · blanc pointillé = extension hors socle.

■ 1. Gestion documentaire

Élément	Valeur
Titre	Dossier d'architecture technique InduSense 4.0 — Sprint 3 CISIA
Version	1.0
Statut	Architecture de référence fonctionnelle
Public	Apprenants, formateur, développeurs, exploitants et évaluateurs
Objet	Décrire l'architecture complète du Sprint 3, ses flux, ses critères de bon fonctionnement et ses preuves d'acceptation
Hors périmètre	Exploitation cloud réelle, haute disponibilité multi-zone, données industrielles réelles, secrets réels, modèle vision opérationnel
Référentiel	Supports et corrigé InduSense des modules 23 à 34

1.1 Historique

Version	Date	Évolution
1.0	04/09/2026	Création du DAT complet, quatre schémas, critères de fonctionnement, traçabilité et matrice de recette

1.2 Convention de lecture

Marquage	Signification
Référence fonctionnelle	Composant obligatoire dans le Sprint 3 complet ; il doit être opérationnel avant recette
Preuve	Résultat observable qui permet de déclarer le composant fonctionnel
Extension	Variante possible hors socle, sans effet sur le contrat métier
Teal / <i>core</i>	Composant fonctionnel obligatoire du socle
Fond bleu pâle / <i>data</i>	Donnée, base, manifeste ou artefact versionné
Fond gris clair / <i>support</i>	Déclencheur, client ou entrée de contexte
Rouge sombre / <i>critical</i>	Gate, refus ou action critique conditionnant la déclaration « prêt »
Trait plein	Flux obligatoire de l'architecture fonctionnelle
Trait pointillé	Extension optionnelle hors socle Sprint 3, notamment Kubernetes

Les cinq classes visuelles sont définies dans les quatre sources afin de conserver une légende commune. La classe *extension* n'est instanciée que dans le schéma global, seul schéma qui représente Kubernetes comme variante hors socle.

■ 2. Résumé exécutif

InduSense est un cas de maintenance prédictive qui transforme des relevés de température, de pression et d'incidents en une probabilité de panne. Le Sprint 3 complet industrialise ce parcours : paquet Python reproductible, gouvernance des données et modèles, contrôles qualité, API FastAPI, persistance idempotente, image Docker, déploiement local, orchestration Prefect, mesure de dérive, supervision Prometheus/Grafana, alertes et procédures de reprise. Le code applicatif et le modèle restent deux artefacts immuables distincts, réunis par un manifeste de release signé.

L'architecture de référence est considérée fonctionnelle lorsqu'elle comprend :

- une API FastAPI servie par Uvicorn sur le port `8000` ;
- un modèle `RandomForestClassifier` sérialisé avec Joblib, versionné, approuvé et signé dans le registre local `model_artifacts`, puis chargé au démarrage sans être incorporé à l'image API ;
- une stack locale qui lance le proxy d'entrée, l'API, PostgreSQL 16, Prefect Server et ses workers dédiés, Prometheus, Alertmanager, Grafana, le Blackbox exporter et les exporteurs internes de drift et d'orchestration ;
- un deployment Prefect d'entraînement publie des candidats puis des modèles `APPROVED`, tandis qu'un deployment distinct score exclusivement avec la release composite `ACTIVE` de son environnement, selon une cadence horaire ou un rejeu borné, et persiste sans doublon ;
- des exporteurs internes de métriques de drift sur `9109` et d'états Prefect sur `9110` ;
- la chaîne CI/CD installe l'environnement verrouillé, exécute les gates, construit des artefacts immuables, signe leur manifeste de release, les promeut et vérifie le déploiement.

Le document décrit cette chaîne comme un système unique : les briques ne sont pas « fonctionnelles » isolément si le flux complet ne traverse pas ingestion, modèle, service, base, supervision et reprise. La recette de fin de document porte donc sur des preuves bout en bout.

2.1 Qualités attendues

Qualité	Mécanisme	Critère de réussite
Reproductibilité	Python 3.13, <code>pyproject.toml</code> , <code>uv.lock</code> , <code>uv sync --locked</code> en CI	même commit, même environnement ; CI bloquée si le verrou est périmé
Testabilité	Pytest, injection du bundle modèle, <code>TestClient</code>	gates automatisées vertes
Portabilité	image multi-stage et Compose	démarrage sur un poste propre
Traçabilité	métadonnées, request ID, prédiction, run Prefect	chaîne de provenance reconstituable
Résilience	retries, idempotence, healthchecks, rollback	reprise sans doublon ni perte
Sécurité	authentification, limites, secrets externes, non-root, TLS	contrôles testés et observables
Observabilité	<code>/metrics</code> , Prometheus, Grafana, drift, alertes	signal → alerte → runbook prouvé
Livraison continue	artefact immuable, registry, promotion, smoke, rollback	version déployée identifiable et réversible

■ 3. Périmètre et hypothèses

3.1 Inclus

- ingestion des trois sources de données du cas InduSense ;
- fabrication de features temporelles sans fuite ;
- entraînement et chargement du modèle tabulaire ;
- contrat HTTP, validation, erreurs, sécurité et observabilité de l'API ;
- orchestration locale Prefect avec serveur, UI, workers train/score dédiés, déploiements et automatisations ;
- persistance PostgreSQL idempotente des prédictions et de leur provenance ;
- conteneurisation et exécution locale multi-services ;
- CI/CD, artefact immuable, promotion et retour arrière ;
- mesure du drift, SLI/SLO, visualisation et alertes ;
- exploitation locale, modes dégradés et procédures de reprise.

3.2 Hors périmètre du socle

- données de production et connexion à un système industriel réel ;
- haute disponibilité multi-zone ou multi-région, autoscaling, service mesh et bascule automatique ; le multi-instance simple sur un site reste admis pour la concurrence et la tolérance locale, sans garantie de haute disponibilité ;
- exploitation Kubernetes/Helm, qui constitue une variante de plateforme ;
- modèle de vision spécialisé, soumis à sa propre validation métier ;
- astreinte humaine et processus organisationnels hors runbook technique.

3.3 Principes d'architecture

- 1. Contrats explicites.** Les schémas Pydantic, codes HTTP, endpoints de santé et artefacts définissent ce qu'un consommateur peut attendre.
- 2. Métier séparé de l'orchestration.** Prefect appelle les fonctions de `src/indusense` ; la logique métier ne dépend pas du moteur d'orchestration.
- 3. Reproductibilité avant vitesse.** Le verrou `uv.lock` et `uv sync --locked` empêchent une résolution implicite et font échouer la CI si le verrou n'est plus cohérent avec le projet. `--frozen` reste réservé aux étapes de construction où le workspace complet n'est pas encore copié.
- 4. Défaillance visible.** Une API vivante mais sans modèle répond `503` sur `/ready` au lieu de simuler un état prêt.
- 5. Idempotence des effets de bord.** La persistance utilise une clé métier et distingue `INSERTED`, `REPLAY_SAME`, `CONFLICT_PAYLOAD` et `FAILED` ; le flow emploie séparément `SKIPPED_NO_NEW_SOURCE` lorsqu'aucune fenêtre nouvelle n'est disponible. Un retry ne crée ainsi ni doublon ni écrasement.
- 6. Observabilité à faible cardinalité.** Les mesures sont agrégées par route, statut, feature ou fenêtre ; les identifiants uniques ne deviennent pas des labels Prometheus.
- 7. Déploiement interchangeable.** Compose constitue le socle local ; la variante Kubernetes conserve les mêmes images, probes et contrats.

■ 4. Vue globale du Sprint 3 complet

Le schéma suivant est la vue de contexte fonctionnelle de référence. Les traits pleins constituent le parcours obligatoire : données versionnées, pipeline orchestré, modèle publié, API persistante, déploiement contrôlé et supervision exploitable. Docker Compose est le plan d'exécution local de référence ; Kubernetes reste une extension optionnelle qui ne change pas les contrats applicatifs.

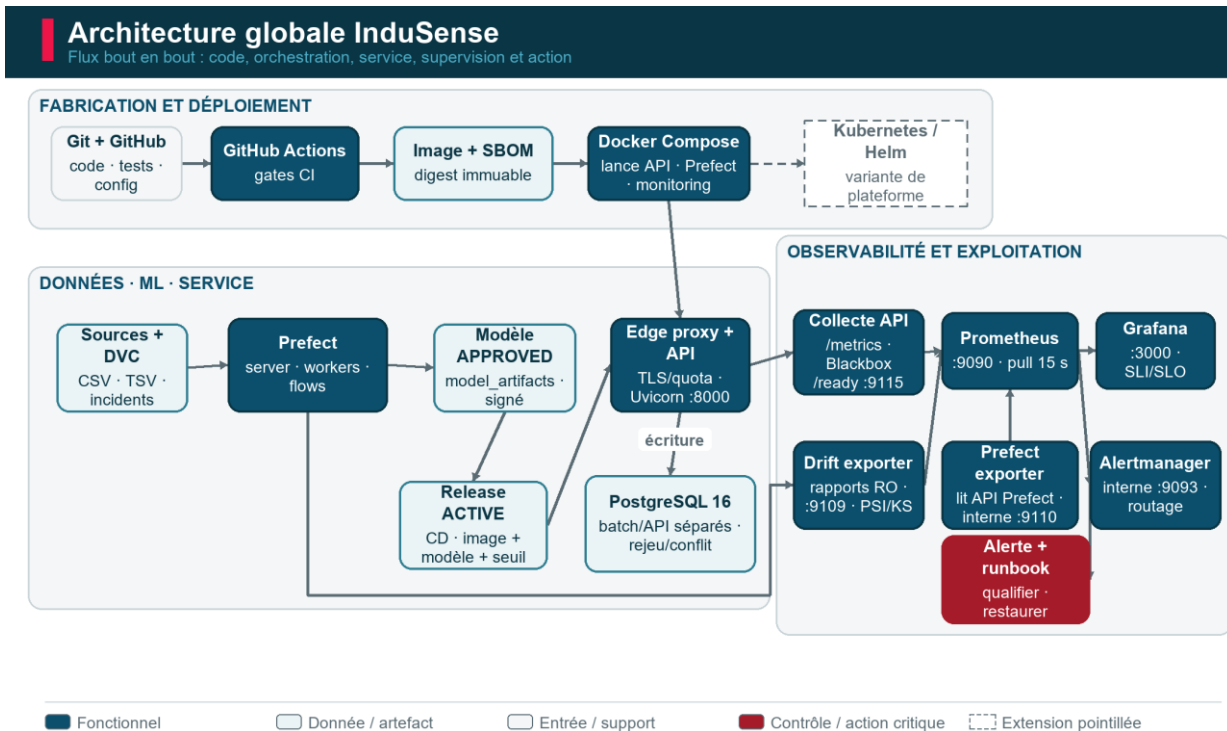


Figure 1 — Vue globale fonctionnelle du Sprint 3 InduSense : edge proxy, modèle APPROVED, release ACTIVE, persistance batch et API séparée, rapports de drift versionnés, sonde readiness, métriques, dashboards et Alertmanager jusqu'au runbook

COMMENT LIRE CE SCHÉMA

Bleu-vert : composant fonctionnel du socle · bleu pâle : donnée ou artefact · gris clair : entrée ou support · rouge sombre : contrôle ou action critique · pointillés : extension optionnelle hors socle.

Dans la figure A4 rendue, les trois liaisons de lancement depuis Compose sont condensées dans « lance API · Prefect · monitoring », l'edge proxy et FastAPI dans « Edge proxy + API », et le scrape /metrics avec la sonde Blackbox dans « Collecte API ». Pour éviter deux longues diagonales, l'entrée de l'image dans la release est portée par « CD · image + modèle + seuil » et le lien Prefect → exporteur par « lit API Prefect ». La source Mermaid éditable jointe au kit conserve toutes les arêtes techniques explicites.

4.1 Lecture de gauche à droite

1. Le développeur versionne code, tests, configuration et références de données dans Git/DVC.
2. La CI reconstruit l'environnement depuis le verrou, exécute les gates et produit un artefact immuable identifié par le commit.
3. Docker Compose déploie le socle local ; healthchecks, readiness et smoke tests valident le démarrage.
4. Prefect Server planifie séparément l'entraînement et le scoring : le premier publie un candidat puis un modèle APPROVED ; le second ne consomme que la release composite ACTIVE de son environnement, puis persiste, produit atomiquement les rapports de drift versionnés et met ceux-ci à disposition de l'exporteur.
5. La CD associe image, modèle, métadonnées, seuil et révision expand dans un manifeste signé portant un `release_id` immuable, puis active atomiquement ce manifeste pour l'environnement. L'API, dont l'image ne contient aucun modèle, et le scoring résolvent ce même pointeur actif.
6. Les prédictions batch sont persistées dans `predictions` par clé temporelle et release ; les réponses API sont isolées dans `api_predictions` par sujet et `Idempotency-Key`. Les deux chemins conservent environnement, `release_id` et provenance sans collision.

7. Prometheus collecte les métriques HTTP, les sondes `/ready`, l'orchestration et le drift via des cibles explicites ; Grafana les visualise et Alertmanager route les alertes vers un runbook testé.
8. Kubernetes peut remplacer Compose si le contexte le justifie, sans modifier les contrats métier.

4.2 Cartographie des composants

Domaine	Composant	Rôle dans le Sprint 3 complet	Critère de fonctionnement
Données	Pandas + DVC	Charger, valider, versionner et fusionner	jeu reproductible, hash et schéma connus
Features	module temporal	Lags 1/3/6 et moyennes 3/6 sans fuite	tests anti-fuite verts
Modèle	scikit-learn + Joblib + registre local <code>model_artifacts</code>	Entraîner, évaluer, approuver, signer et publier un bundle immuable	modèle, métadonnées, seuil et SHA-256 cohérents
API	FastAPI + Pydantic + Uvicorn	Contrat HTTP et inférence	health/ready/predict/metrics testés
Sécurité	middleware + proxy	Authentifier, limiter, tracer et chiffrer	secrets externes, TLS, erreurs contrôlées
Orchestration	Prefect Server + workers train/score	Séparer entraînement/promotion et scoring horaire, reprendre et historiser	workers actifs, deux deployments, schedule et retries prouvés
Persistance	PostgreSQL + repository	séparer écritures batch et API ; <code>INSERTED</code> , <code>REPLAY_SAME</code> , <code>CONFLICT_PAYLOAD</code> ou <code>FAILED</code> ; journal des tentatives	replays batch/API sans doublon, sans collision inter-origine et provenance intacte
Conteneurs	Docker + Compose	Construire et lancer le socle local	tous services healthy
Observabilité	Prometheus + Grafana + Alertmanager	SLI, dashboards, alertes et diagnostics	targets UP, panels alimentés et routage testé
Drift	PSI, KS, métriques modèle	Détecter et qualifier une évolution sur des fenêtres fraîches et closes	seuils, fraîcheur, disponibilité des labels et référence tracés
Exploitation	alertes + runbooks	Transformer un signal en action sûre	alerte injectée et reprise réussie
Livraison	GitHub Actions + registry	Gates, image, promotion, smoke et rollback	release traçable et réversible
Extension	Kubernetes/Helm	Remplacer le plan Compose si nécessaire	hors socle ; mêmes probes et image

4.3 Définition de « tout est fonctionnel »

La plateforme n'est déclarée prête que si la recette prouve simultanément : CI verte et verrou respecté ; image, modèle et manifeste de release identifiés et signés ; migrations Prefect et expand exactes ; services sains ; workers Prefect train/score actifs ; deployments d'entraînement et de scoring validés ; modèle approuvé, release active et seuil cohérent ; couverture de la dernière fenêtre close à 100 % pour la release active ; prédiction valide ; ligne PostgreSQL écrite une seule fois ; contrat de lecture multi-releases déterministe ; cibles Prometheus UP ; métriques fraîches ; dashboards alimentés ; alerte testée ; sauvegarde restaurée ; runbook capable de restaurer le service. L'ouverture d'une interface seule n'est jamais une preuve suffisante.

■ 5. Architecture détaillée de l'API

5.1 Chaîne de traitement d'une prédiction

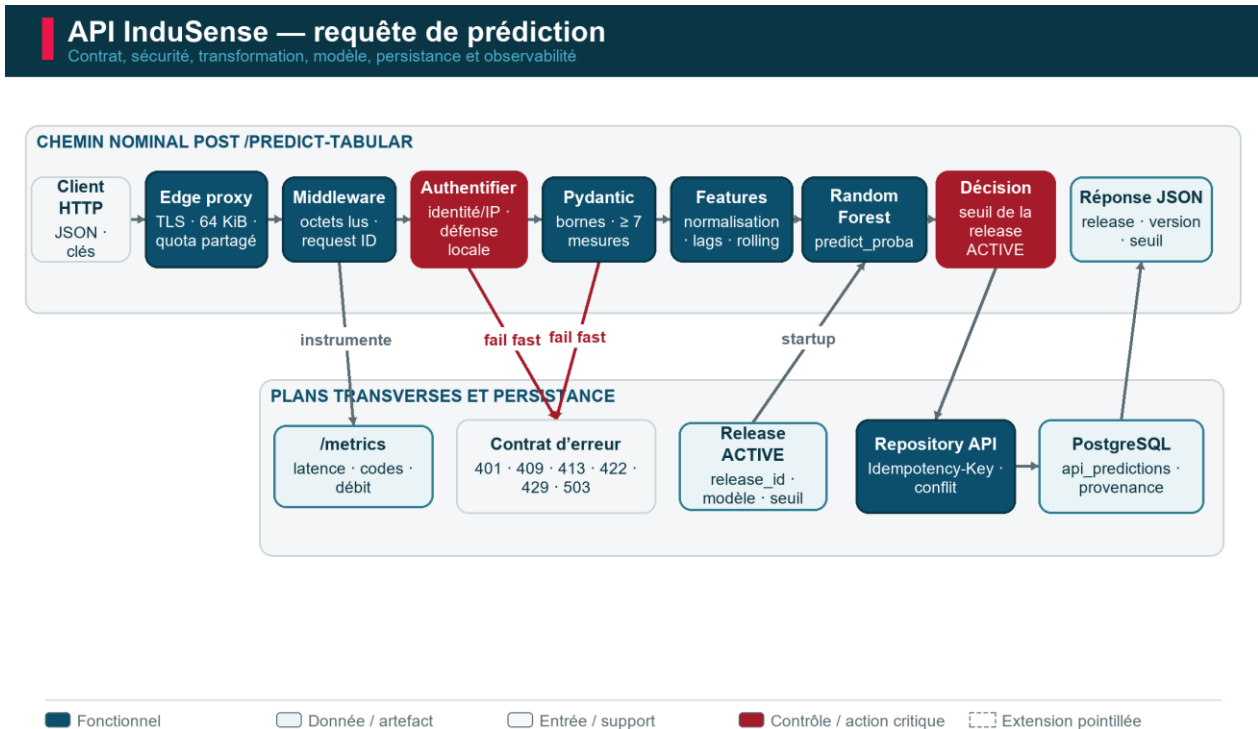


Figure 2 — Chaîne de traitement de l'API InduSense : edge proxy NGINX, contrôles, features, release ACTIVE, Idempotency-Key, persistance interactive séparée et métriques HTTP

COMMENT LIRE CE SCHÉMA

Bleu-vert : composant fonctionnel du socle · bleu pâle : donnée ou artefact · gris clair : entrée ou support · rouge sombre : contrôle ou action critique · pointillés : extension optionnelle hors socle.

5.2 Composants mobilisés

Couche	Outil	Responsabilité	Frontière de preuve
Entrée	NGINX edge proxy	Terminer TLS, écraser les en-têtes de forwarding, limiter le corps et appliquer le quota dans une zone mémoire partagée entre workers NGINX ; l'unique gateway couvre ainsi tous les backends API	nginx.conf, tests réseau et multi-worker
Serveur	Uvicorn	Exécuter l'application ASGI sur 0.0.0.0:8000	Dockerfile
Framework	FastAPI	Routage, injection de dépendances, OpenAPI/Swagger	api/main.py
Contrats	Pydantic	Types, bornes, minimum sept relevés	api/schemas.py
Transformation	Pandas	DataFrame, tri et calcul des features	api/main.py, features/temporal.py
Modèle	scikit-learn	Random Forest 200 arbres, classes équilibrées	models/tabular.py
Sérialisation	Joblib	Charger rf.joblib	api/model_store.py

Couche	Outil	Responsabilité	Frontière de preuve
Persistance	repository SQLAlchemy/Psycopg	Insérer sans écraser, comparer l'empreinte et journaliser la tentative	contrat de base et test d'intégration
Base	PostgreSQL 16	Historiser prédictions, runs et versions	ligne relue après appel API
Configuration	pydantic-settings	Préfixe <code>INDUSENSE_</code> , URI du pointeur de release active, chemins et connexions ; assertion CD optionnelle <code>INDUSENSE_EXPECTED_RELEASE_ID</code> sans pouvoir de sélection	<code>config.py</code>
Logs	Loguru	Corréler les requêtes par <code>X-Request-ID</code>	<code>api/main.py</code>
Métriques	prometheus-fastapi-instrumentator	Exposer <code>/metrics</code>	<code>api/main.py</code>
Sécurité	FastAPI middleware/dependencies	clé, Idempotency-Key, taille réellement lue et plafond local de défense	<code>api/security.py</code>

5.3 Contrat des endpoints

Méthode et route	Authentification	Rôle	Succès	Erreurs principales
GET /health	Non	Liveness : le processus répond	200 {status: ok}	erreur serveur exceptionnelle
GET /ready	Réseau d'administration et sonde interne	Readiness : release active vérifiée, modèle chargé et PostgreSQL joignable	200 + <code>release_id</code> , version, digest et seuil pour CD/Blackbox	403 depuis l'extérieur ; 503 si une dépendance bloquante échoue
POST /predict-tabular	X-API-Key + Idempotency-Key	Prédiction tabulaire persistée et rejouable	200 réponse typée ; rejeu identique retourne la réponse initiale	401, 409, 413, 422, 429, 503
GET /metrics	Réseau de supervision interne	Métriques Prometheus	200 texte OpenMetrics	403 depuis l'extérieur
GET /docs	Réseau d'administration authentifié	Swagger UI générée	200 HTML	403 depuis l'extérieur

5.4 Contrat tabulaire

Entrée. Un `machine_id` et au moins sept objets contenant :

- `timestamp` ISO 8601 avec fuseau explicite, normalisé en UTC ; tout timestamp naïf est rejeté en 422 ;
- `temperature` entre -20 et 200 ;
- `pressure_bar` strictement positive et au plus 400.

Traitement. L'identifiant est normalisé en `MACH-xx`. Les relevés sont triés par machine et par date. Les features sont calculées avec un décalage préalable pour éviter de faire entrer l'observation courante dans sa propre moyenne.

Sortie. `machine_id`, `proba_panne`, `decision`, `release_id`, `model_version` et `threshold`.

5.5 Modèle et features

Le bundle de référence contient un Random Forest de 200 arbres, `class_weight="balanced"` et `random_state=42`. Les métadonnées attestent 1 896 lignes d'entraînement, un taux de panne de 0,1055 et douze features : deux valeurs brutes, six lags et quatre moyennes glissantes. Le seuil de décision est choisi pendant la validation, puis stocké dans `model_metadata.json` avec la version, le schéma de features et le SHA-256 du modèle ; `class_weight="balanced"` ne justifie pas à lui seul un seuil de 0,5.

Famille	Features
---------	----------

AELION Dossier d'architecture technique Sprint 3 · InduSense 4.0	
Famille	Features
Valeurs courantes	temperature, pressure_bar
Retards	temperature_lag1/3/6, pressure_bar_lag1/3/6
Moyennes	temperature_rol13/6_mean, pressure_bar_rol13/6_mean

5.6 Séquence nominale et erreurs

Précondition de démarrage. Avant d'accepter du trafic, le lifespan résout le manifeste `ACTIVE` de l'environnement, vérifie son `release_id`, sa signature, les SHA-256, le schéma, la version, le seuil et la compatibilité de `db_expand_revision`, puis injecte le bundle. Toute absence ou incohérence maintient la readiness interne en `503`.

1. L'edge proxy NGINX refuse les corps annoncés au-delà de 64 KiB, écrase les en-têtes de forwarding et applique dans sa zone mémoire partagée le quota versionné.
2. Uvicorn reçoit la requête admise et FastAPI crée le contexte ; l'application compte les octets réellement lus, y compris en transfert chunked, et le 65 537e octet provoque `413`. Le middleware crée ou reprend ensuite `x-Request-ID`.
3. L'application vérifie la clé, l'`Idempotency-Key` et l'adresse client reconstruite uniquement par le proxy de confiance ; une même clé avec une autre empreinte de requête provoque `409`.
4. Pydantic valide la structure et les bornes.
5. Pandas construit le DataFrame et normalise l'identifiant.
6. Les features temporelles sont calculées ; un historique inutilisable conduit à `422`.
7. Avant l'inférence, l'instance compare son `release_id` local au pointeur `ACTIVE` ; un écart fait passer `/ready` en `503` et refuse la prédiction. Le bundle déjà vérifié au lifespan fournit modèle, métadonnées et seuil.
8. La probabilité de la dernière observation est comparée au seuil.
9. Le repository écrit dans `api_predictions` sous la clé (`environment`, `authenticated_subject`, `idempotency_key`), conserve l'empreinte du payload, le `release_id`, la réponse et `written_at`, puis journalise la tentative séparément. Un rejeu identique restitue la réponse initiale sans réécriture.
10. FastAPI valide et sérialise la réponse ; le request ID est retourné.

ATTENTION — PÉRIMÈTRE

Le contrat fonctionnel du Sprint 3 est `/predict-tabular`. Aucune route d'inférence ne retourne de résultat simulé ; un modèle vision relève d'un produit distinct avec son propre contrat, ses seuils, ses tests et son monitoring.

5.7 Sécurité opérationnelle

- la clé d'API est fournie par un secret externe, rotatable et distinct par environnement ;
- l'unique edge proxy NGINX du socle termine TLS, applique la limite de corps et porte le quota versionné dans une zone mémoire partagée entre ses workers ; puisqu'il voit tout le trafic, ce quota couvre tous les backends API. L'application applique en défense en profondeur un plafond local sur le flux effectivement lu ; une architecture à plusieurs proxys exigerait un magasin distribué, hors socle ;
- les en-têtes de forwarding reçus du client sont supprimés puis reconstruits par le proxy ; l'application ne fait confiance qu'au CIDR de ce proxy ; le profil nominal d'atelier est `60 requêtes/minute` par identité et IP, tandis qu'une identité de recette interne isolée et non utilisable en production reçoit `900 requêtes/minute` afin de laisser une marge contrôlée au test à 10 req/s ;
- `/ready`, `/docs`, `/metrics`, Grafana, Prometheus et Prefect UI ne sont pas exposés publiquement ; `/health` ne révèle qu'un booléen de liveness ;
- le request ID, le sujet authentifié, l'environnement, le `release_id`, la version du modèle et le run sont corrélés dans les logs ;
- le conteneur s'exécute sans privilège, avec système de fichiers en lecture seule sauf volumes explicitement nécessaires ;
- toute erreur d'authentification ou saturation est mesurée sans journaliser le secret ni le contenu sensible.

6. Fonctionnement de Prefect

6.1 Vue de fonctionnement

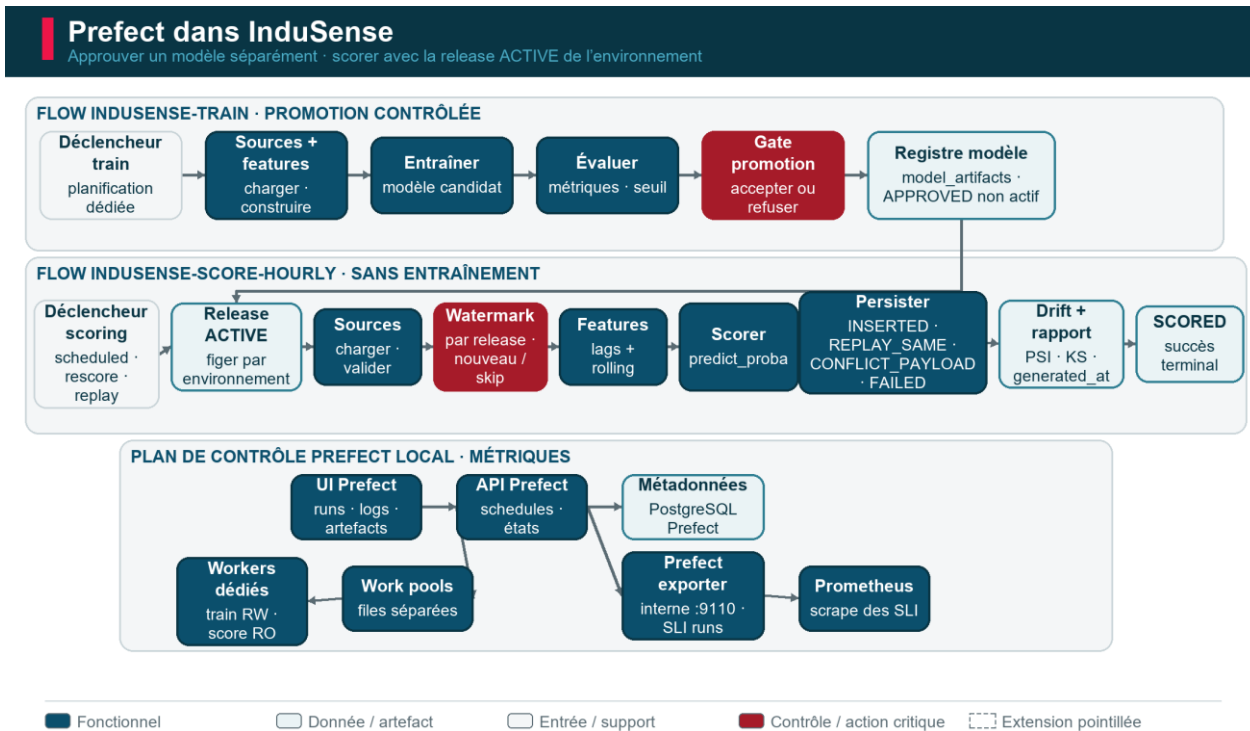


Figure 3 — Prefect : modèle APPROVED après gate, scoring planifié ou rejeu borné qui fige la release ACTIVE, vérifie le watermark, persiste avec les outcomes dont FAILED, publie les rapports puis termine en SCORÉD

COMMENT LIRE CE SCHÉMA

Bleu-vert : composant fonctionnel du socle · bleu pâle : donnée ou artefact · gris clair : entrée ou support · rouge sombre : contrôle ou action critique · pointillés : extension optionnelle hors socle.

Dans la figure A4 rendue, « Workers dédiés » condense les work pools train et score, avec droits RW/RO distincts sur `model_artifacts`, et le nœud Watermark condense le contrôle de progression et le mode de run sans dessiner la branche `SKIPPED_NO_NEW_SOURCE`. Le modèle APPROVED et la résolution de la release ACTIVE restent deux nœuds distincts reliés par la CD ; le flow fige d'abord cette release, charge les sources, vérifie le watermark propre à cette release, puis construit les features. La source Mermaid éditable jointe au kit reste la topologie logique normative.

6.2 Flow fonctionnel de référence

Prefect sépare le contrôle de l'exécution et publie deux deployments à cadence indépendante. Le deployment `indusense-train` produit uniquement des candidats puis, après gate, des modèles éligibles APPROVED ; il ne modifie jamais la release servie. Le deployment `indusense-score-hourly` ne réentraîne jamais : il résout au début du run le pointeur de release ACTIVE de son environnement, vérifie le manifeste signé et utilise ce même `release_id` pour toutes les prédictions du run. Son paramètre `run_mode` est une énumération fermée : `scheduled` traite les seules fenêtres closes postérieures au watermark ; `rescore_release`, déclenché par la CD après activation ou rollback, rejoue une plage bornée et signée ; `manual_replay` rejoue une fenêtre précise avec acteur, justification et identifiant de demande journalisés. `rescore_release` initialise le watermark de la release neuve uniquement après persistance complète, absence de conflit et couverture à 100 % de la dernière fenêtre close. Un rattrapage en échec ne crée ni n'avance ce watermark ; le scoring planifié reste suspendu et ne balaie jamais l'historique. Sa borne de rattrapage est limitée par `INDUSENSE_INITIAL_LOOKBACK_HOURS`, versionnée par environnement et fixée à 24 heures dans le socle. `manual_replay` ne modifie jamais le watermark.

Le label Prometheus `trigger` est distinct du paramètre métier `run_mode` et suit une nomenclature fermée :

Contexte d'exécution	run_mode	trigger exporté	Inclus dans le SLO de succès planifié
----------------------	----------	-----------------	---------------------------------------

AELION		Dossier d'architecture technique		Sprint 3 · InduSense 4.0	
Contexte d'exécution		run_mode	trigger exporté	Inclus dans le SLO de succès planifié	
scoring horaire		scheduled	schedule	oui	
rattrapage après activation ou rollback		rescore_release	deployment_event	non ; suivi par la gate de couverture	
rejeu demandé et tracé		manual_replay	manual	non	
entraînement planifié		sans run_mode de scoring	schedule	oui	
entraînement déclenché par événement		sans run_mode de scoring	deployment_event	non	
entraînement lancé manuellement		sans run_mode de scoring	manual	non	

Les valeurs autorisées de `trigger` sont donc exactement `schedule`, `deployment_event` et `manual`. Pour `indusense_prefect_runs_terminal_total`, `state` vaut exactement `Completed`, `Failed`, `Crashed`, `TimedOut` ou `Cancelled` ; les états non terminaux `Pending` et `Running` ne sont jamais émis dans ce compteur. Le dénominateur du SLO planifié retient uniquement `Completed|Failed|Crashed|TimedOut` avec `trigger="schedule"` ; `Cancelled`, les déclenchements événementiels et les lancements manuels en sont exclus.

Deployment	Task	Entrée	Sortie ou effet	Résilience/gate
Entraînement	charger_sources_entrainement	référence DVC	dataset fusionné	deux retries, délai 5 s
Entraînement	construire_features	dataset	dataset enrichi	calcul déterministe et test anti-fuite
Entraînement	entraîner_candidat	dataset, paramètres versionnés	modèle + métadonnées	aucun effet sur la production
Entraînement	evaluer_candidat	candidat, jeu de validation	métriques, seuil et décision automatique	non-régression, couverture et sécurité bloquantes
Entraînement	publier_candidat	bundle validé	version immuable dans le registre	SHA-256, signature et statut CANDIDATE
Entraînement	promouvoir_modele	candidat éligible + approbation	statut modèle <code>APPROVED</code> + événement d'approbation signé	autorité unique de promotion ; écriture unique et aucune activation implicite
Scoring	charger_release_active	environnement	manifeste, <code>release_id</code> , modèle, métadonnées et seuil	signature vérifiée ; release figée pour tout le run
Scoring	charger_sources_scoring	fenêtre de données et <code>run_mode</code>	dataset fusionné	deux retries, délai 5 s ; bornes obligatoires en rejeu
Scoring	verifier_watermark	clé (<code>environment</code> , <code>deployment</code> , <code>release_id</code>), watermark source et dernier succès complet	poursuivre ou, en mode <code>scheduled</code> , clôturer en <code>SKIPPED_NO_NEW_SOURCE</code>	le watermark n'avance qu'après persistance complète ; rejeu borné et audité
Scoring	construire_features	dataset fusionné	dataset enrichi et indicateurs de qualité temporelle	calcul déterministe, test anti-fuite et mesure de réutilisation
Scoring	scorer_machines	release active, features	machine → probabilité	dernière observation par machine
Scoring	persister_predictions	scores, environnement, release, fenêtre	lignes PostgreSQL + journal de tentatives	<code>INSERTED</code> , <code>REPLAY_SAME</code> , <code>CONFLICT_PAYLOAD</code> ou <code>FAILED</code> par clé composite
Scoring	mesurer_drift	référence, fenêtre, scores	PSI, KS et métriques disponibles	seuils et référence versionnés
Scoring	publier_rapports	métadonnées, scores, drift	Markdown, JSON et CSV	clé stable et run rattaché

`model_version` est attribuée une seule fois lors de la publication du bundle immuable, à partir de l'identifiant du registre et de son SHA-256. Elle n'est dérivée ni du timestamp ni du run Prefect. Le statut `APPROVED` signifie seulement que ce modèle peut entrer dans une release ; le statut `ACTIVE` appartient à une release composite dans un environnement. Le `release_id` est calculé par `sha256` sur la sérialisation JSON canonique du payload `{image_digest, commit_sha, model_version, model_sha256, feature_schema_version, threshold, db_expand_revision}`, donc sans champ `release_id` ni signature. L'enveloppe finale ajoute ce `release_id` au payload, puis signe l'ensemble ; il n'existe ainsi aucune dépendance circulaire. Un candidat refusé reste traçable mais n'est jamais chargé par le scoring ou l'API.

L'activation suit un protocole unique. La CD publie le manifeste signé avec le statut `PENDING`, précharge la release dans un slot hors trafic et vérifie localement ses dépendances, sa `db_expand_revision` et son `release_id`. Elle suspend les nouveaux runs de scoring et draine les requêtes en cours, puis bascule transactionnellement l'unique pointeur `ACTIVE` de l'environnement. Le routeur préconfigure le slot portant ce `release_id` mais maintient son trafic bloqué ; la CD lance immédiatement un `rescore_release` borné sur la dernière fenêtre close, puis n'ouvre le routage qu'après présence de 100 % des machines éligibles dans `predictions_current`, absence de conflit et readiness du slot. Chaque instance compare le pointeur actif avant une inférence ; une ancienne instance encore joignable passe immédiatement `/ready` en `503` et refuse `/predict-tabular` si son `release_id` diffère. Si la bascule, le premier rattrapage ou la surveillance échoue, une seule bascule automatique arrière est autorisée pour ce rollout : la transaction réactive le dernier manifeste sain, le routage reste fermé et un second `rescore_release` vérifie sa couverture avant reprise. Si ce rattrapage de rollback échoue, aucune nouvelle bascule automatique n'est tentée : le pointeur reste gelé sur cette dernière release signée, le scoring planifié et la CD sont suspendus, l'edge reste fermé, l'état terminal `ROLLBACK_FAILED` est déclaré et un incident manuel est requis. Le pointeur transactionnel reste ainsi l'unique autorité, sans oscillation ni fenêtre où deux releases peuvent accepter de nouvelles prédictions.

Deux modes d'usage coexistent :

- **développement/test** : exécution directe et éphémère du flow ;
- **socle fonctionnel** : Prefect Server local, deployments versionnés, work pools et workers train/score dédiés, avec historique persistant.

Dans les deux cas, le code métier reste dans `src/indusense`. Prefect gère la coordination, les états, les logs, les paramètres, les retries et l'artefact ; il ne remplace ni Pandas ni scikit-learn.

6.3 Plan de contrôle et plan d'exécution

Plan	Composant	Responsabilité	Preuve de fonctionnement
Contrôle	Prefect API + UI	déploiements, schedules, états, historique	UI accessible et run retrouvé
Exécution	work pools + workers train/score	recupérer et exécuter les runs dans des files séparées	workers online, runs terminés
Métadonnées	PostgreSQL Prefect	runs, logs, paramètres, artefacts	historique conservé après redémarrage
Code métier	image versionnée	ingestion, features, ML, persistance	image liée au commit
Gouvernance modèle	volume <code>model_artifacts</code> + tables de promotions/activations	bundles immuables signés, approbations et versions servies	historique de promotion, pointeur actif et signature vérifiables
Observabilité Prefect	exporteur interne : <code>9110</code>	traduire l'API Prefect en SLI Prometheus	runs, durée, retard et retries collectés
Secrets	blocks/secret manager	fournir DB, API et stockage	aucune valeur dans Git/logs

DÉCISION D'ARCHITECTURE

Le mode éphémère reste réservé aux tests. Le Sprint 3 complet utilise un serveur Prefect durable, une base de métadonnées sauvegardée et des workers indépendants du poste de développement. L'entraînement/promotion et le scoring sont deux deployments distincts afin qu'un nouvel entraînement ne change jamais implicitement le modèle de production.

La base métier InduSense et la base de métadonnées Prefect sont **deux bases logiques distinctes**, avec utilisateurs, migrations et sauvegardes séparés. Elles peuvent partager une même instance PostgreSQL en

atelier, mais la chaîne de prédiction écrit uniquement dans la base métier ; Prefect conserve de son côté les états, paramètres, logs et artefacts d'orchestration. Chaque service utilise un rôle dédié au moindre privilège, sans droit croisé entre schémas. La durée de conservation des métadonnées Prefect est versionnée par environnement ; les sauvegardes des deux bases portent un horodatage commun et leur restauration coordonnée fait partie de la recette.

6.4 Reprise et idempotence

Un retry n'est sûr que si les tâches avec effet de bord peuvent être rejouées. Les écritures batch et les requêtes interactives sont séparées pour empêcher qu'un historique client et une fenêtre horaire portant sur la même machine entrent en collision :

- table `predictions` réservée au scoring batch, versionnée par migration, avec clé naturelle (`environment`, `machine`, `prediction_ts`, `release_id`) ;
- table `api_predictions` réservée à l'API, avec clé (`environment`, `authenticated_subject`, `idempotency_key`), empreinte du payload et réponse sérialisée ;
- écriture transactionnelle par `INSERT ... ON CONFLICT DO NOTHING`, suivie d'une comparaison de l'empreinte canonique dans la table concernée ;
- colonne `written_at` fixée lors de la première matérialisation et jamais réécrite par un rejeu identique ;
- table append-only `prediction_attempts` qui journalise chaque tentative, son origine `batch` ou `api`, son run/request ID, son empreinte, son horodatage et son résultat parmi `INSERTED`, `REPLAY_SAME`, `CONFLICT_PAYLOAD` ou `FAILED` ; une référence vers `predictions` ou `api_predictions` est portée selon l'origine, avec le sujet et le hash de l'`Idempotency-Key` pour les conflits API ;
- deux exécutions de la même fenêtre en `manual_replay`, à `'release_id' constant`, atteignent toutes deux la persistance et conservent le même nombre de lignes : après un premier passage donnant `N1 > 0`, le rejeu doit produire `N2 = N1`, zéro nouvelle clé métier, une empreinte identique et une tentative supplémentaire marquée `REPLAY_SAME` ; une empreinte différente sur la même clé produit `CONFLICT_PAYLOAD` et aucune mutation de `predictions`.

Le flow publie son résultat `SCORED`, `SKIPPED_NO_NEW_SOURCE` ou `FAILED`, puis les nombres de lignes `INSERTED`, `REPLAY_SAME` et `CONFLICT_PAYLOAD`. Une relance `manual_replay` sur la même fenêtre et la même release doit produire zéro doublon et un résultat métier identique ; sa provenance d'exécution reste visible dans `prediction_attempts` sans réécrire la provenance initiale. Une nouvelle release — y compris lorsqu'elle réutilise le même modèle avec une nouvelle image applicative — crée au contraire une ligne batch distincte par `rescore_release`. Côté API, le client réutilise la même `Idempotency-Key` lors d'un retry : un payload identique restitue la réponse initiale avec le même `written_at`, tandis qu'un payload divergent reçoit `409`. L'ancien commit, l'ancien run ou la réponse initiale ne sont jamais écrasés. Cette conservation de l'historique est intentionnelle et son contrat de lecture est défini en section 7.5.

6.5 Ce que Prefect couvre

Besoin	Fonction Prefect	Exemple InduSense
Ordonnancement	schedule/intervalle/événement	scoring horaire ; entraînement séparé ; rattrapage borné après activation ou rollback
Décomposition	flows et tasks	train → évaluer → approuver ; charger la release <code>ACTIVE</code> → sources → vérifier le watermark → features → score → store → drift → rapports
Autorité de release	pointeur actif par environnement	charger la release <code>ACTIVE</code> → vérifier le manifeste → figer le <code>release_id</code>
Résilience	retries et délais	lecture des sources
Paramétrage	paramètres de run	autre dossier de données
Observabilité	états, logs, durées	historique du pipeline
Restitution	artefacts	rapports Markdown, JSON et CSV
Distribution	workers/work pools	exécution séparée et observable
Automatisation	événements et actions	alerte, <code>rescore_release</code> , relance contrôlée ou suspension

6.6 Configuration versionnée hors interface

- `prefect.yaml` décrit les deployments distincts d'entraînement et de scoring, leurs points d'entrée, paramètres, schedules et work pools ;
- le code du flow, les schedules et les fichiers de déploiement sont versionnés dans Git ; leurs changements passent par pull request, CI et commande de déploiement reproductible ;
- les variables non sensibles sont fournies par environnement ; les secrets passent par des blocks ou un gestionnaire externe ;
- les automatisations sont décrites comme du code ou exportées de façon contrôlée ; l'UI sert à observer et déclencher, pas à être l'unique source de configuration.

■ 7. Données, modèle et persistance

7.1 Sources et fusion

Source	Format	Usage
Température	CSV séparé par ;	mesure capteur
Pression	TSV	mesure capteur
Incidents	CSV	événement cible

Les identifiants machines sont normalisés avant fusion. L'hypothèse du démonstrateur est un relevé unique par machine et par heure. Les lags 1/3/6 et fenêtres 3/6 sont calculés par position ; ils représentent donc des heures uniquement si cette cadence est respectée. La recette contrôle unicité (`machine`, `timestamp`), ordre strict et distribution des deltas. Si le besoin porte sur une durée civile malgré des relevés irréguliers, les séries sont d'abord rééchantillonnées sur une grille horaire avec une politique d'imputation versionnée.

Tous les timestamps d'entrée sont timezone-aware, convertis en UTC avant tri, jointure ou calcul de fenêtre, puis stockés en PostgreSQL sous forme `timestamptz`. Les timestamps naïfs sont rejetés ; `prediction_ts`, `written_at`, watermarks, bornes de fenêtres et dates d'activation sont comparés en UTC afin que les changements d'heure locale ne modifient ni l'idempotence ni `merge_asof`.

`merge_asof` rapproche température et pression dans l'ordre temporel, par machine, avec `direction="backward"` et une tolérance maximale de 90 minutes : une feature ne peut donc jamais utiliser une pression postérieure au timestamp de décision. Le dataset conserve `pressure_source_ts_utc` et calcule `pressure_source_age_seconds`. À cadence horaire, si un point de pression manque, la dernière pression admissible peut alimenter deux décisions consécutives ; ce n'est pas une fuite future, mais une réutilisation susceptible de biaiser PSI/KS. Le pipeline marque `pressure_reused`, publie le ratio de réutilisation par machine et fenêtre et rend les métriques de pression non interprétables au-delà du seuil de qualité versionné de 5 %. Ces champs servent à la qualité et au drift, jamais comme features du modèle. La cible `panne` signale une observation située dans les 24 heures précédant un incident. Le watermark est porté par (`environment`, `deployment`, `release_id`) et n'avance qu'après la persistance complète de toutes les machines éligibles. En mode `scheduled`, une source inchangée produit `SKIPPED_NO_NEW_SOURCE` ; les modes bornés `rescore_release` et `manual_replay` atteignent la persistance sans faire régresser ce watermark.

7.2 Prévention de la fuite temporelle

Toutes les moyennes glissantes appliquent `shift(1)` avant `rolling`. L'observation que l'on cherche à prédire n'entre donc pas dans ses propres features historiques. Le tri par machine et timestamp est une précondition contrôlée.

La validation ML respecte également le temps : l'entraînement, la validation du seuil et le test final utilisent des blocs chronologiques non chevauchants, avec un embargo d'au moins 24 heures entre blocs pour couvrir l'horizon de la cible. Une variante par groupes temporels est admise si toutes les lignes d'une même machine et d'une même fenêtre restent dans un seul fold. Aucun mélange aléatoire ligne à ligne n'est utilisé pour annoncer les métriques de référence.

7.3 Artefacts

Artefact	Rôle	Gouvernance fonctionnelle
rf.joblib	modèle tabulaire dans <code>model_artifacts/<model_sha256>/</code>	version immuable, hash, signature et statut de promotion
<code>model_artifacts/<model_sha256>/model_metadata.json</code>	version, features, seed, volumétrie et seuil	emplacement runtime normatif ; généré, signé et promu avec le modèle
<code>model_artifacts/releases/<release_id>/release-manifest.json</code>	image, modèle, schéma, seuil et <code>db_expand_revision</code> effectivement déployés	signé, immuable et identique entre staging et production
<code>drift_reports/<environment>/<window_id>/<release_id>.json</code>	mesures par fenêtre, hash, schéma, couverture et date de génération	écriture atomique par le score worker ; lecture seule par le drift-exporter
rapports Prefect Markdown/JSON/CSV	restitution humaine et données réutilisables	conservés dans l'historique Prefect et reliés au run
table <code>predictions</code>	décisions et provenance initiale par release	PostgreSQL, migration et rétention définies
table <code>api_predictions</code>	réponses interactives rejouables sans collision avec le batch	clé d'idempotence par sujet, empreinte du payload et réponse initiale conservées
table <code>prediction_attempts</code>	appels, replays et conflits	append-only dans la rétention, origine batch/API, run/request, outcome et empreinte conservés

7.4 PostgreSQL opérationnel

Compose lance une instance PostgreSQL 16 et un script d'initialisation idempotent y crée deux bases logiques, `indusense` et `prefect`, ainsi que des rôles distincts sans droits croisés. Le healthcheck attend le serveur puis vérifie une connexion avec chaque rôle. Le job éphémère `migrate-expand` vérifie le manifeste candidat signé et exécute uniquement `alembic upgrade ${INDUSENSE_EXPAND_REVISION}`, où la valeur doit être exactement sa `db_expand_revision` ; il n'exécute jamais `head`. Le job séparé `prefect-migrate` applique les migrations propres au backend Prefect avec `prefect server database upgrade -y` avant le serveur. Les révisions de retrait incompatibles sont isolées dans le profil manuel `migrate-contract` et ne sont jamais atteignables par le démarrage automatique. `INDUSENSE_DB_URL` est injectée dans `migrate-expand`, `migrate-contract`, l'API, les deux workers métier et `release-manager`, tandis que `PREFECT_API_DATABASE_CONNECTION_URL` est réservée à `prefect-migrate` et au serveur Prefect. Le drift-exporter ne reçoit aucune connexion à la base. Les données des deux bases sont conservées dans le volume `pgdata` et sauvegardées séparément. Un module repository indépendant du framework porte les transactions métier et peut être testé sans serveur HTTP.

Chaque prédiction batch est écrite dans `predictions` avec l'environnement, l'identifiant machine canonique, le timestamp de prédiction UTC, la probabilité, la décision, le `release_id`, la version du modèle, le digest image, le commit, le seuil, le run Prefect, une empreinte canonique et `written_at` en UTC. Une requête interactive écrit séparément dans `api_predictions` le sujet authentifié, l'`Idempotency-Key`, l'empreinte du payload, les mêmes identifiants de release et la réponse sérialisée. Toutes les colonnes temporelles utilisent `timestampz`. `model_promotions` historise le passage d'un modèle de `CANDIDATE` à `APPROVED`. Une table distincte `release_activations` historise, par environnement, le `release_id`, la `model_version`, le digest image, son intervalle d'effet et l'approbation ; une contrainte garantit exactement zéro ou une release `ACTIVE` par environnement.

`prediction_attempts` conserve tous les appels et replays avec leur origine sans modifier la ligne métier initiale. L'API ne renvoie le succès qu'après validation de la transaction, ou utilise une outbox si l'écriture devient asynchrone.

7.5 Contrat de persistance

Le contrat impose :

1. une configuration `db_url` typée ;
2. un module repository indépendant de FastAPI ;
3. des migrations Alembic versionnées pour `predictions`, `api_predictions`, `prediction_attempts`, `scoring_watermarks`, les promotions, les activations et les vues ; la cible automatique est une révision expand exacte ;
4. la clé d'idempotence batch (`environment`, `machine`, `prediction_ts`, `release_id`) ;
5. une clé d'idempotence distincte pour le canal API (`environment`, `authenticated_subject`, `idempotency_key`) et une insertion transactionnelle : conflit identique → réponse initiale/REPLAY_SAME, conflit divergent → CONFLICT_PAYLOAD, 409 et alerte ;
6. des tests d'intégration sur PostgreSQL ;
7. une politique de rétention chiffrée, une purge auditée après sauvegarde et une suspension par legal hold ;
8. une table `model_promotions(model_version, approval_event_hash, status, approved_at, approved_by)` pour l'éligibilité du modèle, avec unicité de (`model_version`, `approval_event_hash`) et rejet de tout événement divergent pour une version déjà approuvée ;
9. une table `release_activations(environment, release_id, model_version, image_digest, effective_from, effective_to, status)` avec unicité de la release active ;
10. une vue `predictions_history` qui expose toutes les releases encore présentes dans la période de rétention, sans écrasement ;
11. une vue `predictions_current` réservée au batch qui, pour chaque (`environment`, `machine`, `prediction_ts`), sélectionne l'écriture réalisée avec le `release_id` actuellement ACTIVE dans cet environnement ;
12. une vue `api_predictions_history` qui restitue les réponses interactives par sujet et clé sans les mélanger à la vérité batch ;
13. une table `prediction_attempts(attempt_id, origin, batch_prediction_id?, api_prediction_id?, authenticated_subject?, idempotency_key_hash?, environment, machine, prediction_ts, release_id, run_or_request_id, payload_hash, outcome, attempted_at)` append-only pendant sa durée de rétention, avec `outcome` ∈ {INSERTED, REPLAY_SAME, CONFLICT_PAYLOAD, FAILED} et une contrainte imposant exactement une référence d'origine lorsque la ligne métier existe ;
14. une table `scoring_watermarks(environment, deployment, release_id, last_source_watermark, last_window_id, updated_at)` initialisée par le `rescore_release` post-activation à la dernière fenêtre close entièrement persistée ; elle n'est ni créée ni avancée si le rattrapage échoue, n'avance ensuite qu'après succès complet et n'est jamais modifiée par `manual_replay` ;
15. un contrat temporel UTC : timestamps timezone-aware en entrée, colonnes `timestamptz`, aucune comparaison sur l'heure locale.

Pour une fenêtre close `W`, une **machine éligible** est une machine déclarée active à `W.end` dans le référentiel versionné `machine_registry(environment, machine, effective_from, effective_to, registry_version)` et disposant, dans `W`, de toutes les mesures sources et features obligatoires selon le contrat de données versionné. Le `registry_version` et `window_id` sont figés dans les paramètres du `rescore_release`. Une machine activée après `W.end` rejoint la prochaine fenêtre entièrement close ; une machine décommissionnée avec `effective_to ≤ W.end` est exclue. Si l'ensemble est vide, le run se termine en `NO_ELIGIBLE_MACHINES`, `indusense_active_release_eligible_machines` vaut 0, `indusense_active_release_coverage_valid` vaut 0 et le ratio de couverture est `N/A`, jamais 1. Ce cas ne ferme pas silencieusement le rollout : il exige la validation explicite du référentiel ou l'attente d'une fenêtre éligible.

Si la release active ne possède pas encore de résultat pour une machine et un timestamp, la lecture courante renvoie explicitement un état « en attente de scoring » et ne retombe jamais silencieusement sur une release obsolète. Toute activation ou tout rollback ouvre un nouvel intervalle d'activation pour une release immuable déjà signée, déclenche immédiatement un `rescore_release` borné et exige une couverture complète avant clôture du rollout ; l'historique reste intact dans sa période de rétention et une activation de staging ne peut jamais influencer une lecture de production.

7.6 Rétention métier

Environnement	predictions et api_predictions	prediction_attempts	Promotions, activations et preuves agrégées de purge
Développement / atelier	90 jours	90 jours	5 ans
Staging	180 jours	90 jours	5 ans
Production	24 mois	12 mois	5 ans

Les tables volumineuses sont partitionnées dans le temps. Une partition close n'est candidate au **DROP** que si elle est expirée **et ne contient aucune ligne de la release actuellement `ACTIVE`** dans son environnement. La présence d'une seule ligne active diffère la partition entière : aucune ligne n'est déplacée ou réécrite pour forcer la purge, la dette de rétention est mesurée et signalée, et la durée nominale peut être dépassée intentionnellement jusqu'à une activation sûre ou une décision approuvée. La purge intervient après sauvegarde conforme, écrit une preuve agrégée signée — bornes, volumes et empreinte — et peut aussi être suspendue par legal hold.

`predictions_history` signifie donc « toutes les releases conservées dans la rétention », jamais « conservation infinie ». Une exigence légale ou contractuelle plus stricte prime par décision approuvée et tracée.

■ 8. Conteneurisation et exécution locale

8.1 Image API

Le Dockerfile suit une construction multi-stage :

- stage `build` sur Python 3.13 slim ;
- copie d'`uv` depuis une image versionnée ;
- installation stricte avec `uv sync --frozen` ;
- stage runtime minimal ;
- suppression de `pip` et `setuptools` ;
- utilisateur non-root UID `10001` ;
- healthcheck HTTP sur `/health` ;
- démarrage Uvicorn sur le port `8000`.

L'image API ne contient aucun fichier modèle. Le socle Compose retient une implémentation unique et testable du registre : le volume nommé `model_artifacts`, organisé par SHA-256, contient les bundles et manifestes immuables signés. Au démarrage, le lifespan résout dans PostgreSQL le pointeur de release `ACTIVE` de son environnement, lit sur ce volume monté en lecture seule le manifeste `model_artifacts/releases/<release_id>/release-manifest.json`, récupère le bundle `APPROVED`, puis vérifie signature, `release_id`, SHA-256, `model_version`, schéma de features, seuil et compatibilité de `db_expand_revision` avant chargement. L'assertion optionnelle `INDUSENSE_EXPECTED_RELEASE_ID`, fournie uniquement par la CD, peut vérifier l'identité attendue mais ne sélectionne ni ne surcharge jamais le modèle, le seuil ou les métadonnées signés. Toute incohérence maintient `/ready` en `503` ; sur le réseau interne d'administration uniquement, la réponse de readiness expose le `release_id` réellement chargé pour permettre à la CD de terminer ou d'annuler le rollout. Le point d'entrée public ne route pas `/ready`.

8.2 Services Compose

Service	Image/construction	Port hôte	Dépendance	Persistance
edge-proxy	NGINX verrouillée par tag et digest	8000/TLS en atelier, 443/TLS hors atelier	API sur le réseau privé	configuration TLS, forwarding, limite 64 KiB et zone de quota partagée montée en lecture seule
api	build local, digest dans le manifeste	non publié	job <code>migrate-expand</code> terminé + release <code>ACTIVE</code>	image sans modèle ; <code>model_artifacts</code> monté en lecture seule
db	<code>postgres:16</code> verrouillée par digest	non publié	aucune	volume <code>pgdata</code> ; bases logiques et rôles <code>indusense/prefect</code> séparés
migrate-expand	même image applicative	non publié	PostgreSQL healthy + manifeste candidat signé	job éphémère ; <code>alembic upgrade \${INDUSENSE_EXPAND_REVISION}</code> sur la révision exacte du manifeste, jamais head
migrate-contract	même image applicative, profil manuel fermé	non publié	24 h de SLO stables + restauration prouvée + approbation	retrait incompatible uniquement sur révision explicitement approuvée ; jamais lancé au démarrage
prefect-migrate	Prefect 3 verrouillée	non publié	base <code>prefect</code> healthy	job éphémère <code>prefect server database upgrade -y</code>
prefect-server	Prefect 3 verrouillée par tag et digest	4200	<code>prefect-migrate</code> terminé	historique des runs
prefect-train-worker	même image applicative	non publié	Prefect Server	<code>model_artifacts</code> en lecture-écriture, uniquement pour publier un bundle immuable
prefect-score-worker	même image applicative	non publié	Prefect Server + <code>migrate-expand</code> terminé + release <code>ACTIVE</code>	<code>model_artifacts</code> et sources en lecture seule ; <code>drift_reports</code> en écriture atomique

Service	Image/construction	Port hôte	Dépendance	Persistance
release-manager	même image applicative, profil CD éphémère	non publié	migrate-expand terminé + image signée + modèle APPROVED	vérifie et installe le manifeste signé, puis bascule le pointeur PostgreSQL ; montage RW limité au job
prometheus	image verrouillée par tag et digest dans <code>images.lock.yml</code>	9090	aucune	configuration et TSDB montées, rétention minimale 35 jours
grafana	image verrouillée par tag et digest dans <code>images.lock.yml</code>	3000	Prometheus	provisioning monté
drift-exporter	image applicative	interne 9109	rapports <code>drift_reports</code> versionnés	volume monté en lecture seule ; aucun accès PostgreSQL ; état dérivé et reconstituable
prefect-exporter	image applicative	interne 9110	API Prefect	curseur de collecte reconstituable
blackbox-exporter	version verrouillée par tag et digest	interne 9115	API /ready	aucune
alertmanager	version verrouillée par tag et digest	interne 9093	aucune au démarrage ; reçoit les alertes de Prometheus	routage d'alertes

Toutes les images tierces sont référencées par tag lisible **et** digest immuable dans `images.lock.yml`. Une mise à jour est une modification explicite de ce fichier, revue et testée ; Compose et la CI vérifient que les images résolues correspondent exactement aux digests déclarés. Un contrôle automatisé de vulnérabilités s'exécute chaque semaine et à chaque build ; une alerte critique ou une version en fin de support ouvre immédiatement une PR de mise à jour. La revue mensuelle des versions supportées déclenche au besoin une PR qui doit repasser verrou, scans, tests, smoke tests et recette de rollback avant modification du digest.

Les volumes nommés du socle sont explicites : `pgdata` pour PostgreSQL, `prometheus_data` pour la TSDB, `model_artifacts` pour le registre de bundles/manifests et `drift_reports` pour les rapports JSON versionnés. Le train worker écrit un nouveau répertoire adressé par empreinte sans modifier les précédents ; le score worker et l'API lisent `model_artifacts`, tandis que seul le score worker écrit atomiquement dans `drift_reports`. `release-manager`, seul job CD autorisé à installer un manifeste dans le volume, vérifie d'abord l'image, le modèle et le manifeste signés, puis met à jour `release_activations` dans une transaction. MLflow peut remplacer cette implémentation via un adaptateur, mais n'est pas une dépendance du socle Compose de référence.

8.3 Réseau

Dans le réseau Compose, `edge-proxy` est l'unique point d'entrée client et relaie vers `api:8000` ; sa zone mémoire de quota est partagée entre ses workers NGINX et couvre tous les backends puisque l'unique gateway voit tout le trafic. L'API, les deux workers Prefect, `migrate-expand`, `migrate-contract` et `release-manager` atteignent la base `indusense` sur `db:5432` avec le rôle minimal propre à leur fonction ; `prefect-migrate` et `prefect-server` atteignent la base `prefect` avec leurs rôles dédiés ; les workers et `prefect-exporter` atteignent `prefect-server:4200`. Le drift-exporter ne reçoit ni rôle, ni URL, ni route vers PostgreSQL : il lit uniquement `drift_reports` en lecture seule. Prometheus scrape `api:8000`, `drift-exporter:9109` et `prefect-exporter:9110`, et délègue la sonde HTTP interne de `/ready` à `blackbox-exporter:9115`. Grafana interroge `prometheus:9090`, et Prometheus envoie ses alertes à `alertmanager:9093`. Les ports applicatifs internes `8000`, `9109`, `9110`, `9115` et `9093` ne sont pas publiés directement. Dans l'atelier, les ports hôte `8000` du proxy et `4200`, `9090`, `3000` des interfaces formateur sont liés uniquement aux interfaces prévues ; la base et les volumes restent privés. En staging et production, `/ready` et les interfaces d'administration sont accessibles uniquement depuis le réseau d'administration authentifié ou les sondes autorisées.

8.4 Amorçage à froid

Un environnement neuf suit une séquence fermée et testable :

1. PostgreSQL démarre, crée les deux bases et les rôles, puis passe healthy ;
2. `prefect-migrate` prépare la base Prefect ; Prefect Server puis le train worker deviennent disponibles ;
3. le premier entraînement publie un bundle immuable ; `promouvoir_model`, autorité unique de promotion, vérifie l'approbation explicite, fait passer une seule fois le bundle à `APPROVED` dans le registre et émet un événement d'approbation signé. Au tout premier bootstrap, cette étape n'exige pas la base métier : après migration,

`release-manager` importe seulement cet événement dans `model_promotions`, idempotemment sous l'unicité (`model_version`, `approval_event_hash`). Le même hash rejoué ne produit aucun nouvel effet ; un hash divergent pour la même version déclenche un conflit et bloque la release. `release-manager` ne décide ni ne réalise jamais la promotion ;

4. la CI/CD construit et signe l'image ainsi que le manifeste candidat complet, comprenant la révision rétrocompatible `db_expand_revision` ;
5. `migrate-expand` vérifie ce manifeste et applique exactement cette révision ; `release-manager`, qui dépend de ce succès, installe le manifeste puis crée la première release `ACTIVE` ;
6. l'API et le score worker démarrent avec cette release ; un `rescore_release` borné peuple la dernière fenêtre close et la gate exige 100 % des machines éligibles sans conflit ;
7. l'edge proxy n'ouvre le trafic qu'après readiness, couverture complète, cibles Prometheus `UP` et smoke test ; Grafana, Alertmanager et les runbooks deviennent alors la preuve d'exploitation.

Une reprise ultérieure réutilise le même ordre et ne fabrique jamais une release implicite. L'échec d'une migration, d'une signature, de la readiness ou de la couverture arrête l'amorçage avant ouverture du trafic.

8.5 Kubernetes

Kubernetes est une variante de plateforme au-delà du socle Compose. Elle reprend les mêmes images et contrats :

- Deployment de l'API avec probes `/health` et `/ready` ;
- Service interne, Ingress TLS et politique réseau ;
- Secret externe ou SecretStore ;
- Job/CronJob ou worker Prefect pour les flows ;
- StatefulSet ou, de préférence, PostgreSQL managé ;
- Prometheus Operator et dashboards provisionnés ;
- rollout contrôlé et rollback.

■ 9. CI/CD et stratégie de promotion

9.1 Chaîne fonctionnelle complète

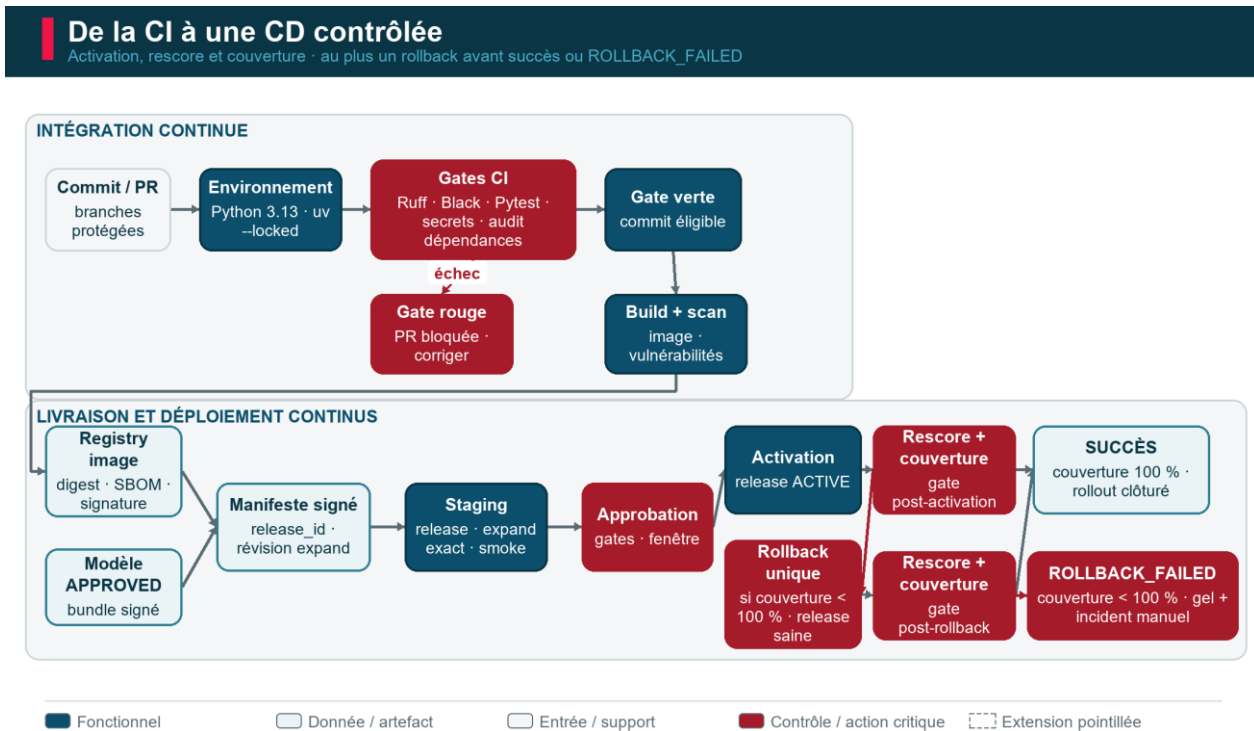


Figure 4 — Chaîne CI/CD : manifeste signé, activation, rescore et couverture, succès ou rollback unique, second rescore et couverture, puis succès ou état terminal ROLLBACK_FAILED

COMMENT LIRE CE SCHÉMA

Bleu-vert : composant fonctionnel du socle · bleu pâle : donnée ou artefact · gris clair : entrée ou support · rouge sombre : contrôle ou action critique · pointillés : extension optionnelle hors socle.

9.2 Gates d'intégration continue

Le workflow se déclenche sur les pull requests et les branches autorisées. Il utilise Ubuntu, Python 3.13 et `uv`, puis exécute :

1. `uv sync --locked --extra dev ;`
2. `uv run ruff check . ;`
3. `uv run black --check . ;`
4. `uv run pytest -q ;`
5. détection de secrets et audit de dépendances ;
6. construction et smoke test de l'image.

Les hooks pre-commit donnent un retour rapide au développeur ; les mêmes contrôles sont répétés dans GitHub Actions, car la CI reste l'arbitre partagé.

9.3 Livraison et déploiement continu

1. L'image est construite une seule fois après validation stricte de `uv.lock`.
2. Le scan, la SBOM et la signature accompagnent son digest ; les images tierces sont vérifiées contre `images.lock.yml`.
3. La CD résout un modèle `APPROVED`, sérialise canoniquement image, commit, modèle, schéma, seuil et `db_expand_revision`, calcule le `release_id = sha256(payload)` avant d'ajouter cet ID et la signature à l'enveloppe finale.

4. Le registre conserve l'image, le bundle et ce manifeste sous des identifiants immuables.
5. Staging déploie exactement cette release composite, fait appliquer par `migrate-expand` la seule révision rétrocompatible signée, puis exécute readiness et smoke tests sans reconstruire l'artefact.
6. Après approbation, la production précharge le même manifeste dans un slot `PENDING`, le vérifie, draine l'ancienne release puis bascule l'unique pointeur `ACTIVE` ; l'API et le scoring ne reprennent qu'avec ce `release_id`.
7. La CD déclenche immédiatement `rescore_release` sur la dernière fenêtre close et attend `indusense_active_release_coverage_valid{environment} == 1`, `indusense_active_release_prediction_coverage_ratio{environment} == 1` et zéro conflit. Le rollout reste ouvert et le trafic du nouveau slot bloqué tant que cette gate n'est pas satisfaite.
8. Les SLI sont surveillés pendant la fenêtre de validation ; tout dépassement critique ou couverture incomplète à la deadline de dix minutes réactive une seule fois le dernier manifeste sain, relance son propre `rescore_release` et maintient le routage fermé jusqu'à cohérence de l'API, du scoring et du `release_id`. L'échec de ce second rattrapage clôt l'automatisme en `ROLLBACK_FAILED`, gèle le pointeur sur cette release signée, suspend scoring/CD et exige une reprise manuelle ; aucune boucle de bascules n'est autorisée.

Les migrations suivent le pattern **expand** → **migrate** → **contract** : la version N ajoute d'abord des structures rétrocompatibles, migre les données, puis ne retire les anciennes structures qu'après expiration de la fenêtre de rollback. L'application N-1 reste donc compatible pendant cette fenêtre. Le job automatique `migrate-expand` exécute `alembic upgrade ${INDUSENSE_EXPAND_REVISION}` sur la cible exacte portée par le manifeste signé et refuse `head`, une valeur absente ou une divergence. Les révisions `contract` sont isolées de cette chaîne ; seul le profil manuel `migrate-contract`, avec une révision exacte, peut les atteindre après au moins 24 heures de SLO stables, une sauvegarde/restauration réussie et une approbation explicite de fermeture. La CI vérifie qu'aucune révision `contract` non approuvée n'est ancêtre de la cible `expand` automatique. Toute migration non rétrocompatible exige une sauvegarde vérifiée, un script de retour et un test de restauration avant promotion. Après `contract`, le rollback de l'application N vers N-1 n'est plus garanti : la reprise suit alors un `restore` vérifié ou une migration corrective vers l'avant.

9.4 Politique de promotion

Étape	Entrée	Gates	Sortie
Validation	commit/PR	lock cohérent, lint, format, tests, secrets	commit éligible
Modèle	bundle CANDIDATE	non-régression, labels suffisants, schéma, seuil et approbation	bundle APPROVED signé
Construction	commit éligible	images verrouillées, scan, SBOM	image signée
Assemblage	image signée + modèle APPROVED	compatibilité, empreintes et signature	manifeste composite signé + <code>release_id</code> candidat
Staging	manifeste composite candidat	révision expand exacte, readiness, tests API et scoring	release candidate vérifiée
Production	approbation	fenêtre, sauvegarde, seuils SLO, activation atomique et readiness sur <code>release_id</code>	release <code>ACTIVE</code> , rollout encore ouvert
Vérification	release active	<code>rescore_release</code> , couverture active à 100 %, absence de conflit, erreurs, p95, ready, drift, fraîcheur	clôture du rollout ou rollback ciblé avec nouveau rattrapage

La preuve de promotion compare les identifiants d'artefact : `digest_build = digest_registry = digest_staging = digest_production`. Elle vérifie aussi que `release_id`, `commit_sha`, `model_version`, `model_sha256`, `feature_schema_version`, `threshold` et `db_expand_revision` sont identiques au manifeste signé et aux valeurs exposées par chaque `/ready`. La CI effectue ces comparaisons après le push registre, après le déploiement staging et avant puis après l'activation production. Toute inégalité bloque ou annule la promotion, ouvre un incident de livraison et restaure l'environnement concerné sur le dernier manifeste sain. Les tags lisibles peuvent évoluer, mais les digests promus restent immuables ; un rollback sélectionne explicitement une release saine,

remet son pointeur actif et son API en cohérence, puis ne se termine qu'après le `rescore_release` et la couverture active complète.

■ 10. Observabilité, drift et exploitation

10.1 Chaîne de supervision

L'API expose les métriques HTTP sur `/metrics`. Dans `monitoring/prometheus.yml`, Prometheus collecte explicitement l'API et les exporteurs toutes les 15 secondes ; le job `blackbox-ready` possède lui aussi `scrape_interval: 15s`, indépendamment de l'intervalle d'évaluation des règles fixé à 30 secondes. Un Blackbox exporter interne sonde `/ready` sans dépendre du scrape `/metrics` et publie `probe_success{target="api-ready"}` ainsi que la durée de la sonde. L'exporteur de drift lit exclusivement les rapports JSON versionnés de `drift_reports` montés en lecture seule, vérifie leur schéma et leur empreinte, ne possède aucun accès PostgreSQL et expose :

- `indusense_drift_psi` ;
- `indusense_drift_ks_pvalue` ;
- rappel, précision, taux d'alerte, taux de panne, PR-AUC et ROC-AUC par fenêtre close lorsque les labels sont disponibles ;
- `indusense_model_metrics_available`, `indusense_labels_coverage_ratio` et le timestamp de la dernière fenêtre étiquetée ;
- `indusense_drift_report_generated_timestamp_seconds`, `indusense_drift_report_age_seconds` et `indusense_drift_export_success`.
- `indusense_pressure_reuse_ratio{environment}` et l'âge de la mesure de pression source, utilisés comme indicateurs de qualité et jamais comme features du modèle. Les rapports conservent le détail machine/fenêtre, mais la série Prometheus est un agrégat glissant sans identifiant de machine ni de fenêtre afin de borner la cardinalité ;
- `indusense_persistence_conflicts_total{environment,origin}` avec `origin` ∈ {`batch`,`api`}, compteur de conflits idempotents sans identifiant unique en label.

La cible métier étant définie sur les 24 heures qui suivent une observation, les métriques supervisées sont calculées uniquement après cet horizon augmenté du délai maximal d'arrivée des incidents, fixé à 2 heures dans le socle. La fenêtre d'évaluation glissante de sept jours se termine donc à `now - 26 h`. Si elle contient moins de 200 observations ou si la couverture des labels est inférieure à 95 %, l'exporteur publie `indusense_model_metrics_available = 0`, conserve les indicateurs de complétude et ne republie pas d'anciennes métriques modèle comme si elles étaient fraîches. Le PSI et la valeur brute du KS peuvent rester publiés sans cible ; toutefois, un KS n'est interprétable comme signal qu'avec au moins 200 observations dans chacune des deux fenêtres.

Un exporteur Prefect interne interroge l'API du serveur toutes les 30 secondes et publie sur `:9110` les compteurs `indusense_prefect_runs_terminal_total{environment,deployment,state,trigger}`, l'histogramme `indusense_prefect_start_lag_seconds{environment,deployment,trigger}`, `indusense_prefect_retries_total{environment,deployment,trigger}` et `indusense_prefect_last_completed_timestamp_seconds{environment,deployment}`. Cette dernière série ne retient que les runs planifiés ; un run manuel ne masque donc jamais un schedule en retard. Les valeurs fermées de `trigger` et `state` sont définies en 6.2 ; les valeurs admises d'environnement et de deployment sont listées dans la configuration, et les identifiants de run et de release ne deviennent jamais des labels.

Grafana utilise Prometheus comme datasource et provisionne quatre vues cohérentes : `indusense-overview` pour `/ready`, p95, débit, 5xx et état des targets ; `indusense-prefect` pour succès, échecs, p95 du retard, durée, retries et âge du dernier run ; `indusense-drift` pour PSI, KS, rappel, taux d'alerte, précision, PR-AUC et ROC-AUC ; `indusense-freshness` pour couverture des labels, date/âge du dernier rapport et succès d'export. Chaque panneau indique sa fenêtre, son unité, son absence de données et le lien vers le runbook associé.

10.2 Seuils de drift

PSI	Interprétation pédagogique
< 0,10	stable / RAS
0,10 à < 0,25	à surveiller
≥ 0,25	dérive forte

Le test KS complète le PSI : $p < 0,05$ constitue un signal uniquement si la référence et la fenêtre courante contiennent chacune au moins 200 observations. Une p-value faible indique que les distributions diffèrent, sans mesurer à elle seule l'importance opérationnelle de l'écart et sans déclencher seule un réentraînement. Les deux mesures sont interprétées avec la taille des échantillons, le régime industriel et les métriques du modèle.

10.3 Signaux à suivre

Domaine	SLI	Exemple de source
Disponibilité	succès de <code>/ready</code>	<code>probe_success{target="api-ready"}</code> via Blackbox exporter
Latence	p95 de <code>/predict-tabular</code>	histogrammes FastAPI
Erreurs	taux 5xx, 401, 422, 429	compteurs HTTP
Débit	requêtes de prédiction par seconde	compteurs HTTP
Modèle	rappel, précision, PR-AUC, ROC-AUC	exporteur drift
Données	PSI et KS par feature/fenêtre	exporteur drift
Qualité temporelle	timestamp source, âge et ratio de réutilisation de la pression	exporteur drift
Labels	couverture, disponibilité et dernière fenêtre close	exporteur drift
Fraîcheur	âge et timestamp du dernier rapport de drift	exporteur drift
Orchestration	runs réussis/échoués, durée, retard, retries	exporteur Prefect :9110
Couverture de release	validité du référentiel, nombre éligible, part et nombre manquant pour la release active sur la dernière fenêtre close	<code>indusense_active_release_coverage_valid{environment}</code> , <code>indusense_active_release_eligible_machines{environment}</code> , <code>indusense_active_release_prediction_coverage_ratio{environment}</code> et <code>indusense_active_release_missing_machines{environment}</code> exposées par l'API à partir de la base métier
Activation de release	instant UTC de la dernière activation ou du rollback	<code>indusense_active_release_activation_timestamp_seconds{environment}</code> exposée depuis <code>release_activations</code> sans <code>release_id</code> en label
Conflits de persistance	collisions idempotentes batch/API	<code>indusense_persistence_conflicts_total{environment,origin}</code>

10.4 Contrôles d'exploitation

Les objectifs de référence sont versionnés dans `monitoring/slo.yaml` :

SLI	Objectif	Fenêtre
disponibilité de <code>/ready</code>	$\geq 99,5 \%$	30 jours glissants
p95 de <code>/predict-tabular</code>	$\leq 500 \text{ ms}$	5 minutes, profil de recette 10 req/s
taux d'erreurs 5xx	$< 1 \%$	5 minutes
runs Prefect réussis	$\geq 99 \%$	7 jours glissants
retard de démarrage Prefect p95	$\leq 5 \text{ min}$	24 heures
fraîcheur du rapport de drift	$\leq 2 \text{ h}$	instantané
couverture de la release active	référentiel valide, 100 %, zéro machine manquante	deadline de rollout : 10 minutes après activation ou rollback ; alerte observable au plus tard à 10 min 30 s

- les SLO et le runbook sont versionnés, approuvés et reliés aux alertes ;
- les règles Prometheus et le routage Alertmanager sont provisionnés avec la stack ;
- l'intervalle d'évaluation est de 30 secondes ; `probe_success{target="api-ready"} == 0` doit persister 2 minutes, et les dépassements de latence, d'erreurs ou de fraîcheur 10 minutes ; `up` surveille séparément la collecte des exporteurs et ne vaut jamais preuve de readiness ;
- la recording rule `indusense:api_readiness_ratio_30d = avg_over_time(probe_success{target="api-ready"}[30d])` s'appuie sur le `scrape_interval: 15s` explicite du job `blackbox-ready` et une rétention Prometheus d'au moins 35 jours. L'état vaut `NO_DATA` sans série, `INSUFFICIENT_WINDOW` si l'observation la plus ancienne a moins de 30 jours moins un intervalle ou si `count_over_time(...[30d])` couvre moins de 99 % des 172 800 échantillons attendus à 15 secondes, et `VALID` seulement sinon. Le ratio contractuel et `indusense:api_readiness_error_budget_consumed_ratio_30d = (1 - indusense:api_readiness_ratio_30d) / 0.005` ne sont évalués qu'en état `VALID` ; une alerte dédiée signale les deux états non valides et la consommation au-delà de 1 signale l'épuisement du budget de 0,5 % ;
- la recording rule `indusense:prefect_success_ratio_7d` divise les hausses de `indusense_prefect_runs_terminal_total{state="Completed"}` par celles de `indusense_prefect_runs_terminal_total{state=~"Completed|Failed|Crashed|TimedOut"}` sur sept jours, uniquement pour `trigger="schedule"` ; les runs manuels, annulés avant démarrage et encore en cours sont exclus du numérateur et du dénominateur ;
- `indusense:prefect_start_lag_p95_seconds_24h` applique `histogram_quantile(0.95, ...)` aux buckets de retard des runs planifiés des dernières 24 heures ; sans run éligible, les deux SLO valent `N/A`, jamais `100 %` ;
- `PrefectScheduleStale` compare l'heure courante à `indusense_prefect_last_completed_timestamp_seconds{environment,deployment}` calculé sur les seuls runs planifiés : seuil `7 800 s` pour `indusense-score-hourly` (deux fenêtres horaires + 10 minutes) et `1 210 200 s` pour un `indusense-train` hebdomadaire (deux périodes + 10 minutes) ; si l'entraînement est exclusivement événementiel, cette règle et son SLO de cadence sont explicitement désactivés ;
- `indusense_drift_report_age_seconds > 7200` pendant 10 minutes déclenche `DriftReportStale` ;
- pendant la fenêtre de contrôle post-activation, la règle `ActiveReleaseCoverageIncomplete` exige `indusense_active_release_coverage_valid{environment} == 1` et teste `(indusense_active_release_prediction_coverage_ratio{environment} < 1 or indusense_active_release_missing_machines{environment} > 0) and on(environment) (time() - indusense_active_release_activation_timestamp_seconds{environment} >= 0) and on(environment) (time() - indusense_active_release_activation_timestamp_seconds{environment} <= 900)` avec `for: 10m`. À 30 secondes d'évaluation, elle devient observable au plus tard à 10 min 30 s ; le contrôleur de rollout applique séparément sa deadline dure de 10 minutes. Un référentiel vide ou invalide produit `NO_ELIGIBLE_MACHINES`, un ratio `N/A` et une alerte dédiée au lieu d'un faux succès. Le `release_id` figure dans les annotations et logs, jamais dans les labels ;
- `indusense_active_release_coverage_valid{environment} == 0` pendant la fenêtre post-activation déclenche `ActiveReleaseEligibilityInvalid` sans délai d'attente, interdit de substituer 1 au ratio absent et maintient le rollout ouvert ;
- `increase(indusense_persistence_conflicts_total{environment=~".+",origin=~"batch|api"}[5m]) > 0` déclenche immédiatement `PersistenceConflictDetected` et bloque toute clôture de rollout concernée ;
- `indusense_pressure_reuse_ratio{environment} > 0.05` pendant 10 minutes déclenche `PressureReuseTooHigh` et rend les métriques de pression non interprétables jusqu'au retour sous le seuil avec rapport frais ;
- une alerte de drift contient la fenêtre, la référence, la feature et le lien vers le runbook ;
- les exporteurs drift et Prefect s'exécutent comme des services supervisés et redémarrables ;
- les identifiants d'administration sont uniques, non triviaux et fournis par secret externe ;
- la disponibilité est prouvée par les sondes, un test synthétique de prédiction et les métriques, pas par l'ouverture du dashboard seule.

10.5 Runbook opérationnel

Symptôme	Vérification	Action sûre	Critère de retour
/health KO	processus/conteneur et logs	redémarrer l'API	/health 200
/health OK, /ready 503	présence et compatibilité du modèle	restaurer artefact connu	/ready 200 + version attendue
Prometheus target DOWN	DNS réseau et /metrics	corriger cible/réseau	target UP
Dashboard vide	datasource et plage de temps	tester PromQL puis provisioning	séries visibles
PSI fort	référence, fenêtre, régime	qualifier la dérive avant retrain	décision documentée

AELION		Dossier d'architecture technique		Sprint 3 · InduSense 4.0	
Symptôme	Vérification	Action sûre	Critère de retour		
Run Prefect échoué	task, logs, paramètres	corriger cause puis relancer idempotent	run Completed sans doublon		
Rapport drift périmé	âge, dernier run et watermark	restaurer le flow ou l'exporteur sans masquer l'ancienne valeur	âge inférieur à 2 h et nouveau timestamp		
Labels insuffisants	couverture et cutoff à 26 h	réparer l'ingestion des incidents ; ne pas forcer les métriques	couverture ≥ 95 %, au moins 200 observations closes		
Couverture active incomplète	machines éligibles, conflits, release et fenêtre close	suspendre le rollout, corriger puis relancer <code>rescore_release</code> ; rollback si le délai de 10 min expire	ratio 1, zéro machine manquante et aucune lecture sur une release obsolète		
Éligibilité absente ou invalide	<code>registry_version</code> , dates d'effet, fenêtre close et compte éligible	corriger/valider le référentiel ou attendre la prochaine fenêtre ; ne jamais forcer le ratio à 1	validité 1, ensemble non vide et gate recalculée sur le référentiel figé		
Conflit de persistance	compteur par environnement/origine, clé métier et empreintes dans le journal	bloquer le rollout ou la requête, préserver les deux preuves et qualifier le producteur ; ne jamais écraser	hausse nulle après correction, replay identique en <code>REPLAY_SAME</code>		
Réutilisation de pression > 5 %	fraîcheur, trous de collecte et détail machine/fenêtre du rapport	réparer l'ingestion ou l'alignement temporel ; ne pas interpréter PSI/KS pression pendant l'alerte	ratio ≤ 5 % et rapport frais		
<code>ROLLBACK_FAILED</code>	pointeur gelé, slot, rattrapage et métriques de couverture	conserver l'edge fermé, suspendre scoring/CD et ouvrir une reprise manuelle à partir d'une release signée	couverture valide à 100 %, readiness 200, validation humaine avant réouverture		
Purge de rétention en échec	partitions closes, sauvegarde, legal hold et preuve agrégée	suspendre la purge, préserver les données et corriger le job	seules les partitions éligibles sont purgées et la preuve signée est conservée		

■ 11. Sécurité et frontières de confiance

11.1 Actifs

- données capteurs et incidents ;
- modèle, métadonnées, seuil de décision et manifeste de release ;
- clé d'API et secrets de base ;
- résultats de prédiction ;
- historique des runs, logs et métriques ;
- code et chaîne de livraison.

11.2 Frontières

1. client → reverse proxy de confiance → API ;
2. API et score worker → volume local `model_artifacts` en lecture seule ; train worker et `release-manager` → même volume avec écriture strictement bornée ;
3. services Compose → PostgreSQL ;
4. Prometheus → endpoints de métriques ;
5. GitHub Actions → registre/environnements cibles ;
6. workers Prefect train/score → infrastructure d'exécution.

11.3 Menaces et contrôles

Menace	Contrôle	Preuve attendue	Réponse
appel non autorisé	X-API-Key, rotation et filtrage réseau	tests 401, journal d'accès sans secret	révoquer, renouveler, analyser les accès
déni de service	plafond 64 KiB au proxy et sur le flux lu, quotas et rate limit partagé	tests 413 chunked, 429 multi-worker et métriques de saturation	limiter, isoler puis augmenter la capacité
fuite de secret Git	scan pre-commit + CI et gestionnaire de secrets	pipeline bloqué sur secret factice	révoquer et purger selon la procédure dédiée
altération du modèle	hash, signature, registre et métadonnées liées	vérification de signature avant promotion	bloquer la release et restaurer le digest connu
exécution privilégiée	UID 10001, système de fichiers en lecture seule, réseau filtré	contrôle d'identité et policy automatisée	arrêter l'instance et recréer depuis l'image saine
usurpation d'adresse client	proxy qui écrase les forwarding headers, CIDR de confiance et compteur partagé	test X-Forwarded-For falsifié sans contournement	bloquer la source et corriger la chaîne de confiance
fuite via interfaces techniques	/ready détaillé et endpoints techniques sur réseaux internes, réseau d'administration et labels à faible cardinalité	accès externe refusé à /ready, /metrics, /docs, Grafana, Prometheus et Prefect UI	couper l'exposition et corriger la configuration
dérive non détectée	PSI/KS, dashboard, règles et Alertmanager	alerte injectée reçue avec runbook	qualifier, suspendre ou réentraîner selon l'analyse

ATTENTION — PRODUCTION

Les valeurs `.env`, mots de passe ou clés réelles ne doivent jamais apparaître dans le dépôt, le DAT, les logs ou les labels. Un secret committé doit être révoqué et remplacé ; l'effacer de la dernière version ne suffit pas.

■ 12. Déploiements et environnements

Environnement	Objectif	Données	Orchestration	Exposition
Développement	coder et tester vite	échantillons locaux	flow direct	localhost
Atelier intégré	exécuter le Sprint 3 complet	jeu pédagogique	Prefect Server + workers train/score	Compose ports 4200/8000/9090/3000
Staging	valider release et migrations	synthétiques/masquées	workers train/score dédiés	ingress privé
Production	servir et superviser	données gouvernées	plusieurs workers applicatifs sur un site, sans garantie HA	TLS, auth, réseau filtré

12.1 Configuration externalisée

Les paramètres variables sont injectés par l'environnement : clé API, URL de base, URI du pointeur de release active, chemins et identifiants. `INDUSENSE_EXPECTED_RELEASE_ID` est une assertion optionnelle fournie par la CD : elle compare la release chargée au pointeur et au manifeste signés, mais ne sélectionne ni ne surcharge jamais le modèle, le seuil ou les métadonnées. Le seuil n'est pas surchargeable : il appartient aux métadonnées signées du bundle référencé par la release. Le dépôt versionne `.env.example`, jamais les valeurs réelles. Un gestionnaire de secrets fournit les valeurs à l'exécution en staging et en production.

12.2 Versionnement

Chaque prédiction exploitable doit être rattachée à :

- la version du code et le commit ;
- la version/hash du modèle ;
- le schéma de données ;
- le seuil de décision ;
- l'horodatage de la fenêtre ;
- le run Prefect ;
- l'environnement de déploiement.

DVC versionne les références de données. Dans le socle Compose, `model_artifacts` porte les bundles immuables signés et PostgreSQL porte leurs métriques, paramètres, promotions et activations ; MLflow reste un adaptateur possible sans changer les contrats. Le runtime ne consomme pas simplement le dernier modèle `APPROVED` : il résout l'unique release `ACTIVE` de son environnement. `model_artifacts/releases/<release_id>/release-manifest.json`, signé, réunit le `release_id`, l'`image_digest`, le `commit_sha`, la `model_version`, le `model_sha256`, la `feature_schema_version`, le `threshold` et la `db_expand_revision` effectivement déployés ; la même identité composite est vérifiée par l'API, le scoring, les migrations expand, staging et production.

■ 13. Tests et critères de recette

13.1 Couverture exigée

Niveau	Exemples	Critère de réussite
Unitaire	normalisation, loaders, features, modèle, drift	calculs déterministes et cas limites couverts
API	contrats, auth, taille réelle du flux, adresse client, rate limit, erreurs	statuts et schémas conformes, y compris corps chunked
Intégration	PostgreSQL, idempotence batch/API, vues multi-releases, UTC, Prefect, rapports drift	deux replays par origine, zéro doublon/collision et lecture déterministe
Stack	démarrage Compose, santé, DNS interne, scrapes	tous services healthy, interfaces privées et targets UP
Charge	scénario Locust versionné à 10 req/s pendant 10 min	$p95 \leq 500$ ms et $5xx < 1\%$
Livraison	build, scan, manifeste composite, staging, rollback	mêmes digests image/modèle promus puis restaurés
Reprise	sauvegarde et restauration isolée des deux bases	schémas, historiques et vues vérifiés après restauration

13.2 Recette documentaire et architecturale

- tous les composants dessinés disposent d'un critère de preuve vérifiable ;
- les ports concordent avec Compose et les fichiers de monitoring ;
- les quatre schémas utilisent la même légende ;
- aucune clé, mot de passe réel ou donnée sensible n'est inclus ;
- le PDF est lisible en A4 et à l'écran ;
- les schémas restent utilisables séparément en PNG/SVG ;
- le DOCX conserve titres, tableaux, légendes et texte alternatif ;
- le Markdown contient les sources Mermaid éditables.
- `images.lock.yml` couvre chaque image tierce par tag et digest immuable.

La QA distingue les tableaux normatifs issus du Markdown des tableaux de mise en page générés pour la couverture, le sommaire, la légende et les encadrés. Les en-têtes répétés correspondent exactement aux tableaux normatifs et toutes les lignes sont marquées non sécables ; les comptes finaux sont recalculés après chaque régénération et joints au manifeste de contrôle.

13.3 Recette de bout en bout

1. amorçage à froid depuis des volumes vides, migrations Prefect et expand exactes, première release active et gate de couverture ;
2. contrôle de cadence, watermark par release, modes bornés, fusion temporelle, labels clos et prévention de fuite ;
3. tests d'intégration PostgreSQL, conflits idempotents, rétention et vues multi-releases ;
4. tests `/ready`, `/metrics`, `/predict-tabular`, taille réelle du flux et request ID ;
5. test de la chaîne de confiance proxy, du rate limit partagé et de l'inaccessibilité externe des interfaces techniques ;
6. exécution séparée des deployments d'entraînement/promotion et de scoring ;
7. scan de secrets et dépendances, vérification du verrou d'images, build et signatures ;
8. déploiement staging du manifeste composite et comparaison des digests ;
9. test de charge contre les SLO chiffrés ;
10. métriques Prefect, couverture active, fenêtre SLO, fraîcheur et alertes testées par injection de panne ;
11. validation du schéma et du hash des rapports de drift lus par l'exporteur sans connexion base ;
12. sauvegarde/restauration isolée de PostgreSQL métier et du backend Prefect ;

13. rollback indépendant d'une version modèle et/ou application, suivi d'un rattrapage complet ;
14. revue sécurité et protection des interfaces d'administration.

13.4 Matrice de recette exécutable

Contrôle	Exécution reproductible	Résultat attendu
Authentification	<code>uv run pytest -q tests/test_api_security.py -k missing_key</code>	appel sans clé rejeté en 401, aucune valeur sensible dans les logs
Limite du corps	<code>uv run pytest -q tests/test_api_security.py -k "body_limit or chunked"</code>	corps Content-Length ou chunked supérieur à 64 KiB rejeté en 413 dès le 65 537e octet
Limite de débit	<code>uv run pytest -q tests/test_api_security.py -k "rate_limit or forwarded"</code>	X-Forwarded-For falsifié ignoré, dépassement partagé contrôlé en 429 sur plusieurs workers
Interfaces privées	lancer <code>tests/test_network_exposure.py</code> depuis le réseau externe de recette	/ready, /metrics, /docs, Grafana, Prometheus et Prefect UI refusés ; accès interne/CD/sonde autorisé selon rôle
Validation Pydantic	<code>uv run pytest -q tests/test_api_contract.py -k invalid_payload</code>	corps invalide rejeté en 422 avec erreur typée
Exécution non-root	<code>docker compose exec -T api id -u</code>	UID retourné : 10001
Verrou des images	<code>uv run python scripts/verify_image_lock.py images.lock.yml</code>	chaque image tierce de Compose possède un tag et un digest exacts
SBOM et signatures	<code>cosign verify --key cosign.pub <image>@sha256:<digest> et cosign verify-blob --key cosign.pub --signature release-manifest.sig release-manifest.json</code>	signatures valides, SBOM non vide et manifeste rattaché aux mêmes digests
Migration Prefect	sur base Prefect vide, exécuter <code>docker compose run --rm prefect-migrate</code> deux fois	les deux passages <code>prefect server database upgrade -y</code> réussissent et le serveur démarre sur le schéma attendu
Migration expand rejouable	avec un manifeste candidat signé, exécuter deux fois <code>docker compose run --rm migrate-expand</code>	les deux passages appliquent exactement <code>INDUSENSE_EXPAND_REVISION = db_expand_revision</code> , jamais head, et la version reste identique
Contract non autorisé	rendre une révision contract disponible sans ouvrir le profil ni fournir les preuves/approbation	le démarrage automatique reste sur la cible expand ; la révision contract n'est pas appliquée et la CI prouve qu'elle n'est pas atteignable
Démarrage à froid	démarrer depuis des volumes vides avec un jeu et des artefacts de recette signés	bases/rôles → migrations → Prefect/train → modèle APPROVED → manifeste/migrate-expand/release-manager → release ACTIVE → API/score → <code>rescore_release</code> → couverture 1 → edge et monitoring prêts
Cadence, UTC et fusion	<code>uv run pytest -q tests/test_data_temporal_contract.py</code>	timestamps naïfs rejetés, conversion et stockage UTC, unicité/ordre par machine, delta horaire ou resampling explicite, jointure backward à 90 min maximum, aucune mesure future ; <code>pressure_source_ts_utc</code> , <code>pressure_source_age_seconds</code> , le marqueur et le ratio de réutilisation sont exacts et absents des features modèle
Prévention de fuite temporelle	<code>uv run pytest -q tests/test_features_temporal_leakage.py tests/test_model_temporal_split.py</code>	<code>shift(1)</code> précède chaque <code>rolling</code> ; folds chronologiques sans chevauchement, embargo effectif ≥ 24 h ; un mélange aléatoire ligne à ligne fait échouer le test
Watermark et modes	activer une release sans watermark, exécuter son <code>rescore_release</code> borné puis <code>scheduled</code> et <code>manual_replay</code> ; répéter avec un rattrapage en échec	seul le rattrapage complet à couverture 1 initialise le watermark à la dernière fenêtre close ; <code>scheduled</code> ne balaie jamais au-delà des 24 h configurées ; échec et <code>manual_replay</code> ne créent, n'avancent ni ne font régresser le watermark
Promotion modèle	<code>uv run pytest -q tests/test_model_promotion.py</code>	candidat refusé jamais servi ; <code>promouvoir_modele</code> écrit une fois APPROVED et l'événement signé ; import identique par <code>release-manager</code> sans effet, hash divergent en conflit ; modèle approuvé inactif jusqu'à l'activation atomique

AELION Dossier d'architecture technique		Sprint 3 · InduSense 4.0
Contrôle	Exécution reproductible	Résultat attendu
Séparation train/score	<code>uv run pytest -q tests/test_prefect_deployments.py tests/test_compose_mounts.py</code>	deux deployments à cadences et work pools indépendants ; train seul possède <code>model_artifacts</code> RW, score et API sont RO ; toute écriture interdite échoue
Cloisonnement PostgreSQL	<code>uv run pytest -q tests/integration/test_postgres_roles.py</code>	chaque rôle accède uniquement à sa base et son schéma autorisés ; lectures/écritures croisées métier ↔ Prefect sont refusées explicitement
Cohérence du seuil	altérer le seuil ou le hash du bundle de recette puis démarrer l'API	<code>/ready</code> reste en 503 ; le seuil de réponse est celui du bundle signé après restauration
Idempotence batch	lancer deux <code>manual_replay</code> bornés sur le même snapshot, le même environnement et le même <code>release_id</code> , puis comparer clé, empreinte et <code>written_at</code>	<code>N1 > 0</code> , <code>N2 = N1</code> , zéro doublon, contenu et <code>written_at</code> inchangés, tentative <code>REPLAY_SAME</code> ajoutée ; une empreinte divergente produit <code>CONFLICT_PAYLOAD</code> , une nouvelle release crée une ligne distincte
Idempotence API et isolation	rejouer la même requête avec la même <code>Idempotency-Key</code> , puis la même clé avec un payload différent et enfin un score batch au même timestamp	rejeu identique : même réponse/ <code>written_at</code> ; payload divergent : 409 ; le score batch s'insère sans collision avec <code>api_predictions</code>
Alerte de conflit	provoquer un <code>CONFLICT_PAYLOAD</code> batch puis API et évaluer les règles	le compteur augmente avec <code>origin</code> correct, <code>PersistenceConflictDetected</code> est <code>Firing</code> , le rollout reste bloqué et aucune ligne initiale n'est modifiée
Qualité pression	injecter plus de 5 % de réutilisations sur un rapport frais puis revenir à 5 % ou moins	<code>PressureReuseTooHigh</code> passe à <code>Firing</code> après au plus 10 min 30 s ; PSI/KS pression sont marqués non interprétables, puis redeviennent interprétables avec le rapport frais conforme
Lecture multi-releases	activer R2, exécuter son <code>rescore_release</code> , puis réactiver R1 et exécuter le sien avant d'interroger les vues dans staging et production	<code>predictions_history</code> conserve R1/R2 dans la rétention ; <code>predictions_current</code> suit l'unique release active avec couverture 1, sans fallback ni contamination croisée
Rétention métier	injecter des partitions expirées et non expirées, une partition contenant une ligne de la release active et un legal hold, puis exécuter la purge après sauvegarde	seules les partitions closes, expirées et sans ligne active ni legal hold sont purgées ; la partition active entière est différée sans réécriture, la dette est signalée et la preuve agrégée signée est conservée 5 ans
Contrat des rapports drift	fournir un rapport valide, puis un rapport au schéma ou hash altéré, avec tout accès DB désactivé pour l'exporteur	le rapport valide est exposé ; l'autre est refusé et mesuré ; aucune connexion PostgreSQL n'est tentée
Labels clos	<code>uv run pytest -q tests/test_model_metrics_labels.py</code>	cutoff à 26 h ; métriques modèle absentes si <code>n < 200</code> ou couverture < 95 %, disponibilité à 0
Drift volontaire	<code>uv run python scripts/evaluate_drift.py --reference <ref> --current <courant></code>	PSI/KS produits ; la fenêtre dérivée franchit le seuil configuré
Fraîcheur du drift	figer le rapport tout en laissant l'exporteur actif	âge supérieur à 7 200 s et <code>DriftReportStale</code> à <code>Firing</code> après au plus 10 min 30 s
SLO succès Prefect	injecter des compteurs bornés par état/trigger puis évaluer <code>indusense:prefect_success_ratio_7d</code>	seuls les runs planifiés terminaux entrent au dénominateur ; $\geq 99\%$ passe, < 99 % alerte ; zéro run donne N/A et non 100 %
SLO retard Prefect	injecter des buckets de retards connus puis évaluer <code>indusense:prefect_start_lag_p95_seconds_24h</code> et <code>PrefectScheduleStale</code>	$p95 \leq 300$ s passe ; dépassement alerte ; stagnation après 7 800 s pour le scoring horaire ou 1 210 200 s pour l'entraînement hebdomadaire
Readiness synthétique	rendre la base ou le bundle indisponible tout en laissant le processus et <code>/metrics</code> actifs	<code>/health</code> reste 200, <code>/ready</code> passe 503, up reste 1, <code>probe_success{target="api-ready"}</code> vaut 0 et l'alerte est <code>Firing</code> après au plus 2 min 30 s
SLO disponibilité 30 jours	tester successivement zéro donnée, 10 jours, 30 jours complets et 30 jours avec plus de 1 % de lacunes	<code>NO_DATA</code> , <code>INSUFFICIENT_WINDOW</code> , <code>VALID</code> , puis <code>INSUFFICIENT_WINDOW</code> ; ratio et budget ne sont contractuels qu'en état <code>VALID</code>

Contrôle	Exécution reproductible	Résultat attendu
Couverture post-activation	figer le référentiel et la fenêtre, tester ajout/retrait de machine, ensemble vide, puis rendre une machine éligible absente après activation	l'éligibilité suit les dates d'effet ; l'ensemble vide donne <code>NO_ELIGIBLE_MACHINES</code> et N/A ; sinon <code>ActiveReleaseCoverageIncomplete</code> passe à <code>Firing</code> au plus tard à 10 min 30 s, le rollout ne se ferme pas et le rollback avec rattrapage est déclenché
Routage Alertmanager	injecter l'alerte de recette et vérifier le receiver de test, l'inhibition, le silence puis l'escalade	acheminement vers le receiver de test, règles d'inhibition/silence effectives, runbook rattaché
Charge et SLO	utiliser l'identité interne de recette au quota 900/min ; le <code>locustfile.py</code> impose <code>constant_throughput(1)</code> par utilisateur, puis lancer <code>uv run locust -f tests/load/locustfile.py --headless -u 10 -r 10 -t 10m --csv reports/load --host <url></code>	après warm-up, débit observé entre 9,5 et 10,5 req/s, <code>p95 ≤ 500 ms</code> et <code>5xx < 1 %</code> sans 429 parasite ; incapacité à tenir le débit = échec ; le profil nominal 60/min reste testé séparément
Sauvegarde/restauration	restaurer les dumps métier et Prefect dans deux bases isolées puis lancer les contrôles d'intégrité	versions de schéma, promotions, historiques, runs et vues conformes ; originaux inchangés
Release promue	comparer manifeste, build, registre, révision expand, pointeur actif, API et scoring, puis attendre la couverture	égalité exacte du <code>release_id</code> , des digests image/modèle, du schéma, du seuil et de <code>db_expand_revision</code> ; couverture 1, zéro machine manquante, aucun rebuild ni split-brain
Rollback	pendant la fenêtre ouverte, provoquer l'échec de la release puis du rattrapage de la release saine	une seule bascule arrière est réalisée ; succès : couverture active 1 avant réouverture ; second échec : <code>ROLLBACK_FAILED</code> , pointeur gelé, edge fermé, scoring/CD suspendus et aucune oscillation
Rollback modèle seul	<code>uv run pytest -q tests/test_release_model_only_rollback.py</code>	un nouveau manifeste composite signé associe le digest de l'image courante au modèle sain antérieur ; aucun artefact ou manifeste actif n'est muté, puis activation et rattrapage suivent le protocole normal
Reproductibilité ML	relancer dans le même digest d'image avec mêmes données, lock, seed 42 et paramètres	empreintes données/lock/paramètres/features/modèle strictement identiques ; écart absolu des métriques $\leq 10^{-6}$
Reproductibilité des schémas	exécuter <code>python build_dat.py</code> dans deux copies propres ; comparer les SHA-256 des 4 PNG + 4 SVG	8/8 hashes identiques ; mapping statique → Mermaid ou condensation complet

■ 14. Risques et mesures de maîtrise

Priorité	Risque	Mesure préventive	Preuve et réponse
P0	modèle absent, altéré ou incompatible	bundle et manifeste signés, compatibilité et readiness strictes	503 contrôlé puis restauration du dernier manifeste sain
P0	écritures dupliquées, collision batch/API ou provenance écrasée lors d'un retry	clé batch par environnement/release, clé API par sujet/ <i>Idempotency-Key</i> , tables séparées, outcomes non ambigus et journal append-only dans la rétention	replays par origine, lignes et <i>written_at</i> stables, <i>REPLAY_SAME</i> visible, conflit API divergent en 409
P0	fuite temporelle dans les features	tri, <i>shift(1)</i> avant <i>rolling</i> , tests dédiés	test anti-fuite bloquant la CI
P1	release défectueuse ou couverture incomplète après activation	staging, smoke tests, promotion des mêmes digests, <i>rescore_release</i> et gate à 100 %	rollback testé vers la dernière release composite saine, puis rattrapage complet
P1	dérive non détectée ou mal interprétée	référence et fenêtres versionnées, fraîcheur, PSI + KS + métriques closes	alerte qualifiée puis suspension ou réentraînement
P1	secret divulgué	gestionnaire de secrets, rotation, scan Git et CI	révocation immédiate et analyse de l'exposition
P1	saturation API ou file Prefect	quotas, timeouts, limites de concurrence et backoff	test Locust 10 req/s, SLO p95/5xx, puis réduction de charge/scale-out
P1	supervision aveugle	scrapes, règles, Alertmanager et test synthétique	injection d'incident et exécution du runbook
P2	perte des historiques ou purge excessive	sauvegardes PostgreSQL métier et backend Prefect, rétention chiffrée, legal hold et preuve agrégée	restauration périodique dans deux bases isolées, puis contrôle des schémas, vues, partitions et runs

14.1 Ordre de réponse à incident

1. protéger les personnes, les données et les secrets ;
2. arrêter la propagation : suspendre le deployment, le flow ou la release fautive ;
3. restaurer le dernier manifeste de release connu ; si seul le modèle doit revenir en arrière, activer un manifeste composite déjà construit et signé qui associe l'image courante au modèle antérieur — ne jamais muter un composant d'une release active ;
4. vérifier readiness, prédiction synthétique, persistance et scrapes ;
5. reprendre progressivement l'exécution ;
6. documenter cause, impact, décision et action préventive.

■ 15. Décisions d'architecture

ID	Décision	Justification	Conséquence
ADR-001	FastAPI + Pydantic	contrat et documentation automatiques	validation centralisée, OpenAPI
ADR-002	Uvicorn ASGI derrière un proxy	servir FastAPI et terminer TLS en bordure	probes et timeouts explicites
ADR-003	modèle chargé au lifespan	éviter les I/O à chaque requête	readiness dépend du bundle
ADR-004	features décalées avant rolling	empêcher la fuite temporelle	premières lignes non exploitables
ADR-005	Prefect séparé du métier	garder les fonctions testables	orchestration remplaçable
ADR-006	persistance séparée du batch et de l'API	empêcher les collisions entre fenêtre horaire et historique client tout en rendant chaque retry sûr	clé temporelle/release pour <code>predictions</code> , <code>Idempotency-Key</code> par sujet pour <code>api_predictions</code> , <code>outcomes</code> explicites et <code>journal append-only</code> dans la rétention
ADR-007	Docker multi-stage non-root	réduire taille et privilèges	gestion d'artefacts stricte
ADR-008	Prometheus pull + Grafana	supervision standard et découplée	endpoints et cardinalité à gouverner
ADR-009	Compose comme référence Sprint 3	reproductibilité locale	Kubernetes conserve les mêmes contrats comme extension
ADR-010	pointillés réservés aux extensions	distinguer le socle obligatoire des variantes	aucun doute sur le parcours de recette
ADR-011	image API indépendante du modèle	permettre promotion et rollback modèle sans rebuild applicatif	bundle externe immuable chargé et vérifié au lifespan
ADR-012	manifeste de release composite signé et activation atomique	identifier sans ambiguïté image, modèle, schéma, seuil et révision expand	API et scoring consomment le même <code>release_id</code> actif ; activation/rollback imposent un rattrapage couvert
ADR-013	entraînement et scoring séparés	empêcher un réentraînement de changer implicitement la production	deux deployments Prefect et gate d'approbation
ADR-014	seuil porté par le bundle	garantir que le seuil servi est celui validé avec le modèle	aucune surcharge par environnement ; incohérence en 503
ADR-015	historique multi-releases et vue courante	conserver commits, runs et décisions dans la rétention tout en offrant une lecture opérationnelle unique	activation par environnement explicite, jamais de fallback silencieux, couverture complète avant clôture
ADR-016	watermark par environnement, deployment et release	permettre le scoring planifié et le rejeu contrôlé sans contradiction	modes <code>scheduled</code> , <code>rescore_release</code> et <code>manual_replay</code> , watermark avancé seulement après succès complet

■ 16. Traçabilité pédagogique modules 23 à 34

Module	Objet	Composants/preuves	Chapitres DAT	Compétence dominante
M23	paquet Python propre	pyproject.toml, src/, tests	4, 7, 9, 13	C6
M24	qualité et CI	uv, Ruff, Black, Pytest, GitHub Actions	9, 13, 15	C6/C8
M25	API REST	FastAPI, Pydantic, Uvicorn, Swagger	4, 5, 13	C7
M26	sécurité API	clé, taille, rate limit, threat model	5, 11, 13	C2/C7
M27	conteneur	Docker multi-stage, non-root, healthcheck	8, 11, 13	C6/C7
M28	stack locale	Compose, API, PostgreSQL, Prometheus, Grafana	4, 8, 10, 13	C7
M29	design d'orchestration	flow/tasks Prefect	6, 15	C7
M30	exploitation du flow	retries, schedule, artefact, idempotence	6, 7, 13	C7/C8
M31	concepts de drift	PSI, KS, référence et fenêtres	7, 10, 14	C8
M32	rapport et alerting	rapport JSON/CSV, métriques modèle, TP alerte	10, 13	C8
M33	observabilité	/metrics, Prometheus, SLI/SLO	5, 10, 13	C8
M34	dashboards/runbooks	Grafana, exploitation et preuve	10, 13, 14	C8/C9

Les compétences mobilisées sont interprétées conformément au référentiel du parcours :

Compétence	Définition synthétique appliquée au Sprint 3
C2	Identifier et maîtriser les risques éthiques, sociétaux, réglementaires et de sécurité liés à l'exploitation de la solution IA.
C6	Implémenter le modèle et intégrer les briques techniques, le reporting et le suivi des prédictions dans l'environnement retenu.
C7	Contribuer à concevoir et évaluer l'architecture cible en tenant compte des contraintes et des performances attendues.
C8	Mesurer les performances et les impacts afin de maintenir la solution fonctionnelle conformément aux usages et enjeux.
C9	Inscrire la solution IA dans une amélioration continue tenant compte des besoins, des données et des exigences initiales.

■ 17. Glossaire

Terme	Définition
API	Interface par laquelle un système expose un contrat à d'autres systèmes
ASGI	interface Python asynchrone entre serveur web et application
Bundle modèle	ensemble immuable formé du modèle, de ses métadonnées, de son schéma de features et de son seuil
CI	intégration continue : reconstruire et vérifier automatiquement
CD	livraison/déploiement continu : promouvoir un artefact jusqu'à un environnement
DVC	outil de versionnement des données et artefacts volumineux, relié aux commits Git
Digest	empreinte cryptographique immuable qui identifie exactement un artefact
Drift	évolution des données ou de la relation entre données et cible
Idempotence	propriété d'une opération rejouable sans multiplier son effet métier
Liveness	preuve que le processus est vivant
Manifeste de release	document signé qui lie une image, un commit, un bundle modèle, un schéma et un seuil sous un <code>release_id</code>
Readiness	preuve que le service est prêt à servir correctement
PSI	Population Stability Index, mesure d'ampleur d'un décalage de distribution
KS	test statistique de comparaison de deux distributions
MLflow	plateforme de suivi des expériences, métriques, paramètres et versions de modèles
Outbox	table transactionnelle d'événements à publier de façon fiable après une écriture métier
PR-AUC	aire sous la courbe précision-rappel, utile notamment lorsque la classe positive est rare
ROC-AUC	aire sous la courbe taux de vrais positifs/taux de faux positifs sur tous les seuils
SLI	indicateur mesuré de niveau de service
SLO	objectif associé à un SLI et une fenêtre
Work pool	abstraction Prefect reliant des déploiements à des workers
Conflit idempotent	sur une clé existante, ne rien modifier si l'empreinte est identique et échouer explicitement si elle diverge
SBOM	inventaire des composants logiciels d'un artefact
Watermark	dernier timestamp source complètement persisté, porté par environnement, deployment et release ; il ne régresse jamais lors d'un rejeu

■ 18. Sources et provenance

Le DAT consolide les supports des modules 23 à 34, le corrigé InduSense et les contrats techniques ci-dessous. Ces références servent à retrouver les exemples, les conventions de données et les preuves de recette ; le présent document demeure la vue d'architecture fonctionnelle de bout en bout.

Sujet	Source locale
API et endpoints	<code>src/indusense/api/main.py</code>
Schémas Pydantic	<code>src/indusense/api/schemas.py</code>
Chargement modèle	<code>src/indusense/api/model_store.py</code>
Sécurité	<code>src/indusense/api/security.py</code>
Configuration	<code>src/indusense/config.py</code>
Données et features	<code>src/indusense/data/loaders.py</code> , <code>src/indusense/features/temporal.py</code>
Modèle	<code>src/indusense/models/tabular.py</code> ; runtime normatif <code>model_artifacts/<model_sha256>/model_metadata.json</code> et <code>model_artifacts/releases/<release_id>/release-manifest.json</code> ; <code>artifacts/models/model_metadata.json</code> n'est qu'une fixture pédagogique de dépôt, jamais la source runtime
Registre local et releases	volume <code>model_artifacts</code> , module de registre, <code>release-manager</code> , tables <code>model_promotions</code> et <code>release_activations</code>
Prefect	deployments d'entraînement/scoring, <code>prefect.yaml</code> , exporteur et scripts de recette
Conteneurs	<code>Dockerfile</code> , <code>docker-compose.yml</code> , <code>images.lock.yml</code>
Migrations	<code>alembic.ini</code> , <code>migrations/versions/</code> , jobs Compose <code>migrate-expand</code> , <code>migrate-contract</code> et <code>prefect-migrate</code>
CI	<code>.github/workflows/ci.yml</code> , <code>.pre-commit-config.yaml</code>
Drift	<code>src/indusense/monitoring/drift.py</code> , <code>scripts/evaluate_drift.py</code>
Supervision	<code>monitoring/prometheus.yml</code> , <code>monitoring/slo.yml</code> , exporteurs drift/Prefect, règles Alertmanager et provisioning Grafana
Charte	<code>parcours_CISIA/Sprint3/BRIEF_design_AELION.md</code>

18.1 Livrables associés

- `DAT_InduSense_Sprint3_v1.0.md` — source documentaire Markdown et quatre blocs Mermaid éditables ;
- `build_dat.py` — générateur reproductible du DOCX et de quatre vues statiques de présentation ;
- `schemas/01_architecture_globale_indusense`, `02_api_indusense_detaillée`, `03_prefect_indusense` et `04_ci_cd_indusense` — quatre vues PNG/SVG et leurs sources Mermaid `.mmd` normatives, extraites exactement des blocs du Markdown ;
- `DAT_InduSense_Sprint3_v1.0.docx` et `DAT_InduSense_Sprint3_v1.0.pdf`.

Les `.mmd` constituent la topologie logique normative et sont identiques aux blocs Mermaid du Markdown après normalisation des fins de ligne. Les PNG/SVG ne sont pas des rendus directs de Mermaid : `build_dat.py` produit des vues A4 condensées et déterministes dont les regroupements et arêtes fusionnées sont expliqués sous chaque figure. Dans le manifeste et la QA, « correspondance » signifie donc correspondance sémantique documentée, pas identité de graphe ni de bytes ; `mermaidExactAfterNormalization` vérifie uniquement Markdown ↔ `.mmd`. La recette reconstruit les vues statiques dans deux répertoires propres, compare leurs hashes et vérifie que tout élément statique se rattache à un élément normatif ou à une condensation explicite.

Ces éléments représentent seize fichiers contrôlés : un Markdown, un générateur, un DOCX, un PDF, quatre sources `.mmd`, quatre PNG et quatre SVG. `MANIFESTE_SHA256.txt` est l'index d'intégrité qui les référence ; il n'est volontairement pas auto-haché et ne compte pas parmi les seize.